第 7 讲　人工神经网络

Artificial Neural Network（ANN）

从人脑神经元到人工神经网络

神经网络基本概念（样本输入、输出格式）

组合、激励函数

感知机+实际应用

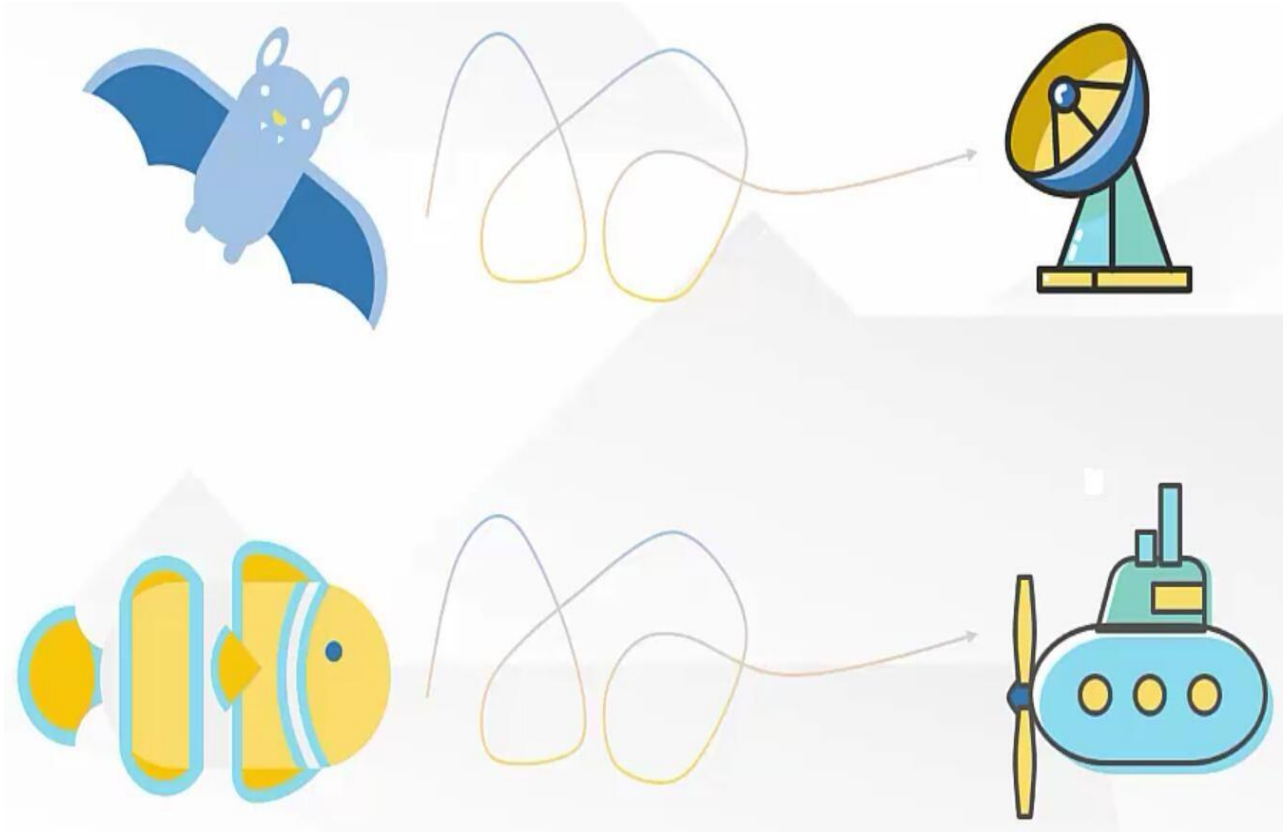
梯度下降和delta法则、反向传播学习算法

BP算法神经网络

神经网络模型的优势和缺点

什么是仿生学

仿生学：通过分析生物系统的特质，再通过工程技术进行实现并有效利用生物功能的一门学科



蝙蝠 —— 雷达

鱼 —— 潜水艇

鱼鳍 —— 桨

鲨鱼 —— 泳衣

鸟 —— 飞机

. . .

. . .

. . .



联结主义学派

- **又称仿生学派或生理学派**
  - 认为人的思维基元是神经元，而不是符号处理过程
  - 认为人脑不同于电脑
- **核心：**智能的本质是联接机制。
- **原理：**神经网络及神经网络间的连接机制和学习算法



麦卡洛可（McCulloch）



皮茨（Pitts)

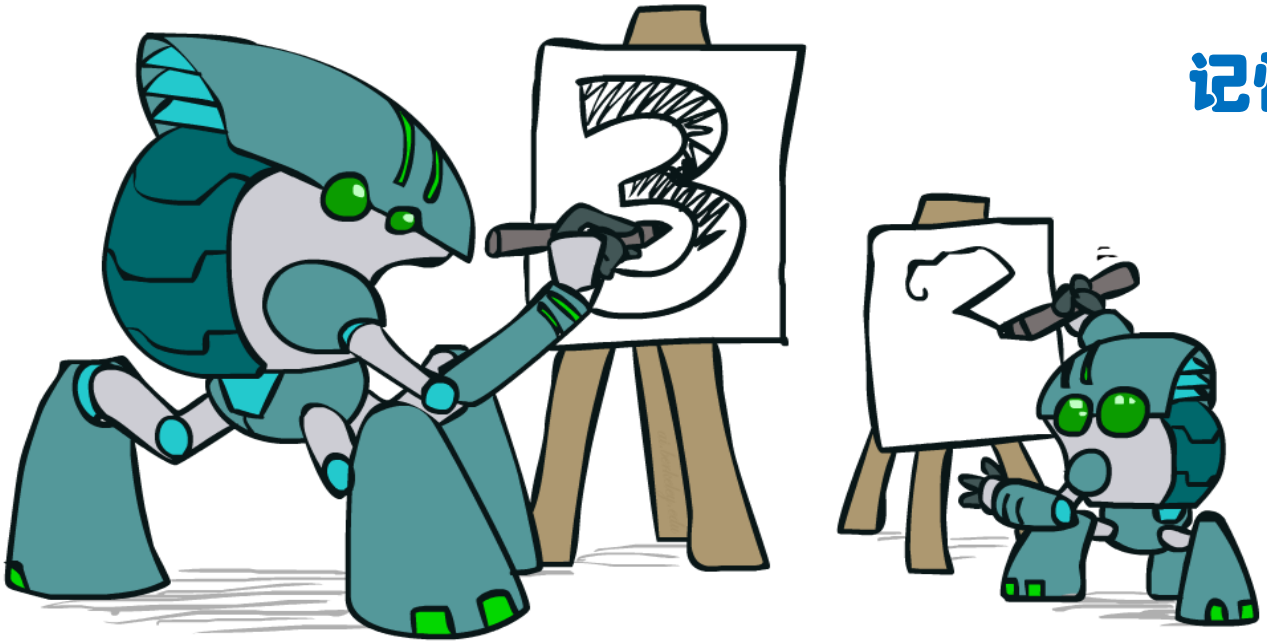
- 10^{11}: The number of neurons in the human brain
- 10^{4}: The average number of connections of each neuron
- 10^{-3}: The fastest switching time of neurons
- 10^{-10}: The switching speed of computers
- 10^{-1}: The time required to visually recognize your mother



Biological Motivation

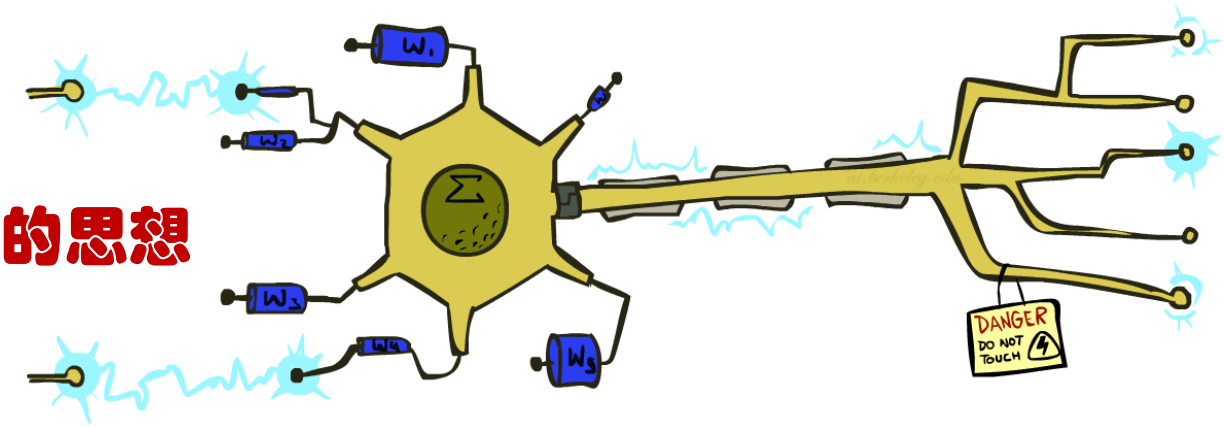
- The power of parallelism（并行计算）
- The information processing abilities of biological neural systems follow from highly **parallel** processes operating on representations that are **distributed** over many neurons.（并行计算+分布式存储）
- The motivation of ANN is to capture this kind of highly parallel computation based on distributed representations.（动机=捕获基于分布式表示的高度并行计算的能力）
- **Group A**（理解生物学习过程）
  - Using ANN to study and model biological learning processes.
- **Group B**（获得机器学习算法建模分类器）
  - Obtaining highly effective machine learning algorithms, regardless of how closely these algorithms mimic biological processes.

生物学启示



记忆力-学习能力太强

过拟合、
过学习



完全是机器学习的思想



什么是人工神经网络?

- 人工神经网络（*Artificial Neural Networks*，*ANNs*）的简称
- 人工神经网络(*ANN*)的研究在一定程度上受到了生物学的启发，因为生物的学习系统(如大脑)是由相互连接的神经元(*Neuron*)组成的异常复杂的网络。由于ANN只是在一定程度上受生物神经系统的启发，所以ANN并未模拟生物神经系统中的很多复杂特征
- ANN主要是模拟人脑的神经元分布式和并行计算的能力
- ANN是由一系列简单的单元相互密集连接构成的:
  - 每一个单元有一定数量的实值输入(可能是其他单元的输出)
  - 每一个单元产生单一的实数值输出(可能成为其他很多单元的输入)
- ANN领域的研究者分为两个团体:
  - 一个团体的目标是 使用ANN研究和模拟生物学习过程(本身)
  - 另一个团体的目标是 获得高效的机器学习算法（应用），不管这种算法是否反映了生物过程



人工神经网络应用

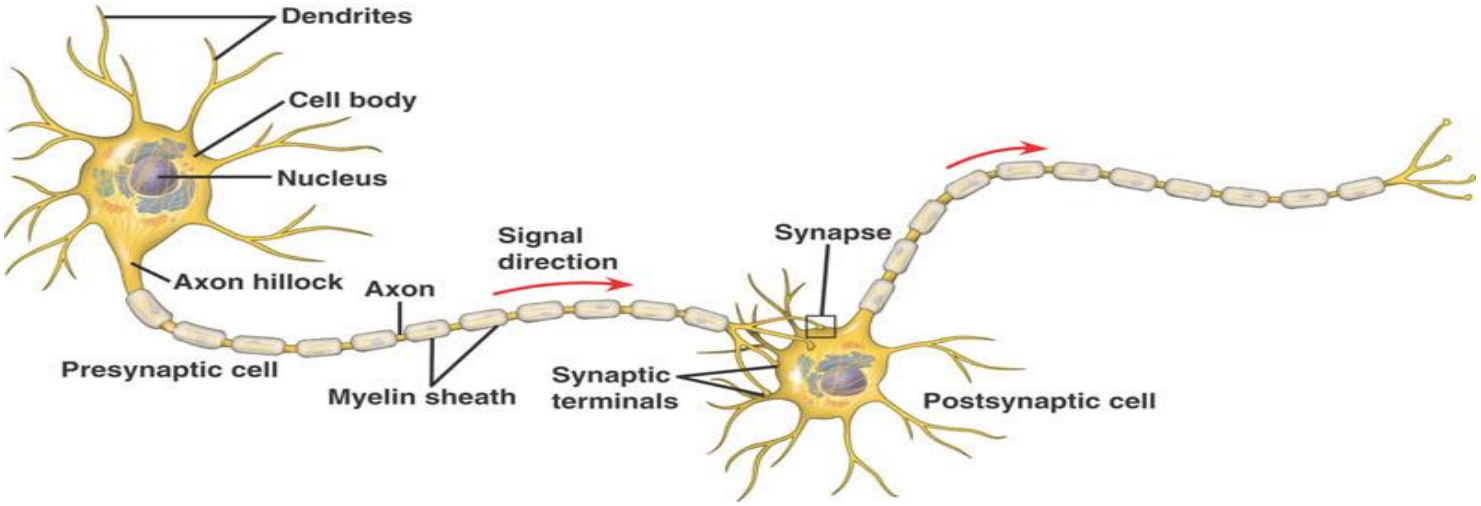
- 神经网络学习方法对于逼近实数值、离散值或向量值的目标函数提供了一种健壮性很强的方法。
- 在模式识别领域，如学习解释复杂的现实世界中的传感器数据，人工神经网络是目前知道的最有效的学习方法。比如，学习识别手写字符、学习识别口语等。
- 常用的反向传播算法（BP），使用梯度下降来调节网络参数，以最佳拟合由输入输出对组成的训练数据，在很多实际的问题中取得了惊人的成功。



适合神经网络学习的问题

- **实例是用很多“属性一值”对表示的**：要学习的目标函数是定义在可以用向量描述的实例之上的，向量由预先定义的特征组成。
- **目标函数的输出**：可能是离散值、实数值或者由若干实数属性或离散属性组成的向量。
- **训练数据可能包含错误**：ANN学习算法对于训练数据中的错误有非常好的健壮性。
- **可容忍长时间的训练**：网络训练算法通常比像决策树学习这样的算法需要更长的训练时间。
- **人类能否理解学到的目标函数是不重要的**：神经网络方法学习到的权值经常是人类难以解释的。学到的神经网络比学到的规则难以传达给人类。

生物神经元组成



- 神经元组成：细胞体（*Cell Body*），轴突（*Axon*），树突（*Dendrites*），突触（*Synapse*）
- 神经元之间通过 突触 两两相连，信息的传递发生在 突触 。
- 突触记录了神经元间联系的强弱
- 只有达到一定的兴奋程度，神经元才向外界传输信息

神经元

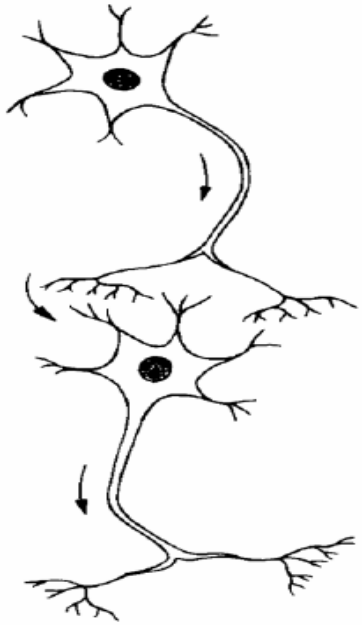
- **神经元特性**
  - **信息以预知的确定方向传递：**

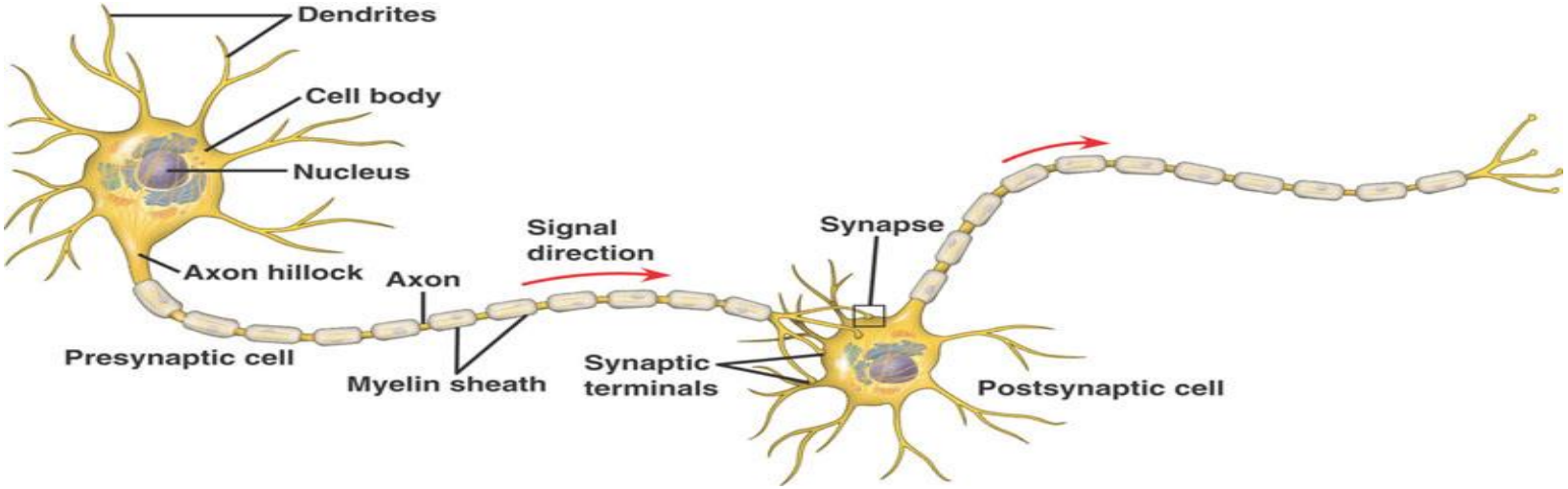
    一个神经元的树突 ▶▶ 细胞体 ▶▶ 轴突 ▶▶ 突触 ▶▶ 另一个神经元树突

  - **时空整合性：**

    对不同时间通过同一突触传入的信息具有时间整合功能

    对同一时间通过不同突触传入的信息具有空间整合功能







神经元

- **工作状态**
  - 兴奋状态，对输入信息整合后使细胞膜电位升高，当高于动作电位的阈值时，产生神经冲动，并由轴突输出。
  - 抑制状态，对输入信息整合后使细胞膜电位降低，当低于动作电位的阈值时，无神经冲动产生。

- **结构的可塑性**
  - 神经元之间的柔性连接：突触的信息传递特性是可变的

    ——学习记忆的基础



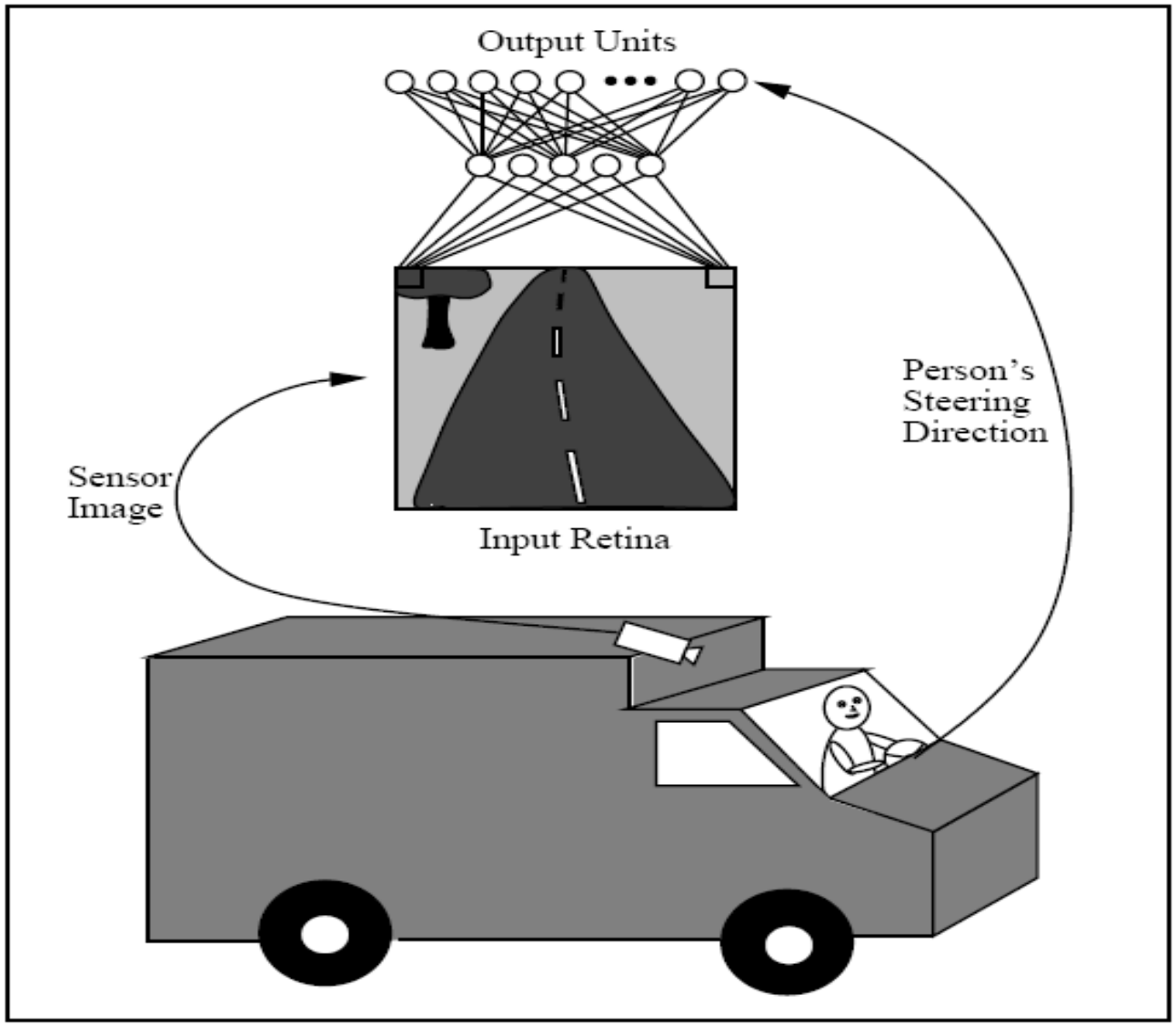
生物系统中的学习

- 自适应学习
  - 适应的目标是基于对环境信息的响应获得更好的状态
- 在神经层面上，通过突触强度的改变来实现学习
  - 消除某些突触，建立一些新的突触
- Hebb学习律
  - 神经元同时激活，突触强度增加
  - 异步激活，突触强度减弱
- 学习律符合能量最小原则
  - 保持突触强度需要能量，所以在需要的地方保持，在不需要的地方不保持。





Case Study


Output Units
Person's
Steering
Direction
Sensor
Image
Input Retina



ANN的学习方法

- **通过神经网络所在环境的模拟过程，调整网络中的自由参数**

  基本思想：*Learning by data*

- **学习模型**

  批学习 *vs.* 间隔依次随机学习（*Batch vs. Incremental*）

- **两种类型**

  Supervised vs. Unsupervised



ANN的学习规则

- 能量最小 （*ENERGY MINIMIZATION*）
- 对人工神经网络，需要确定合适的能量定义
- 可以使用数学上的优化技术来发现如何改变神经元间的联接权重。

ENERGY* = *Measure of Task Performance Error

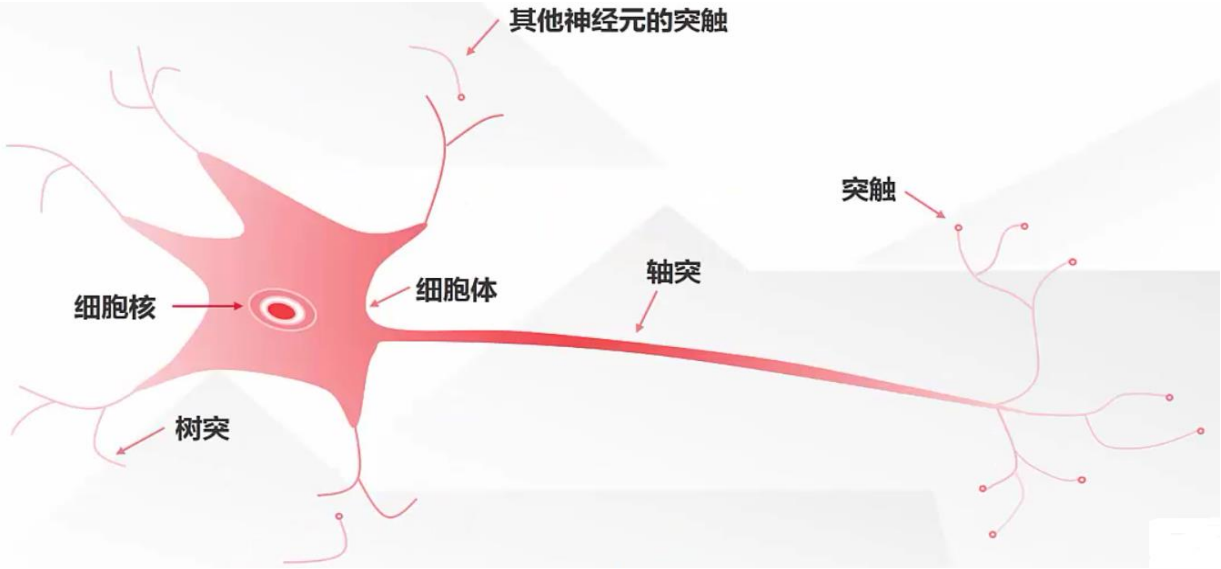
***MSE*（*Mean Square Error*）**

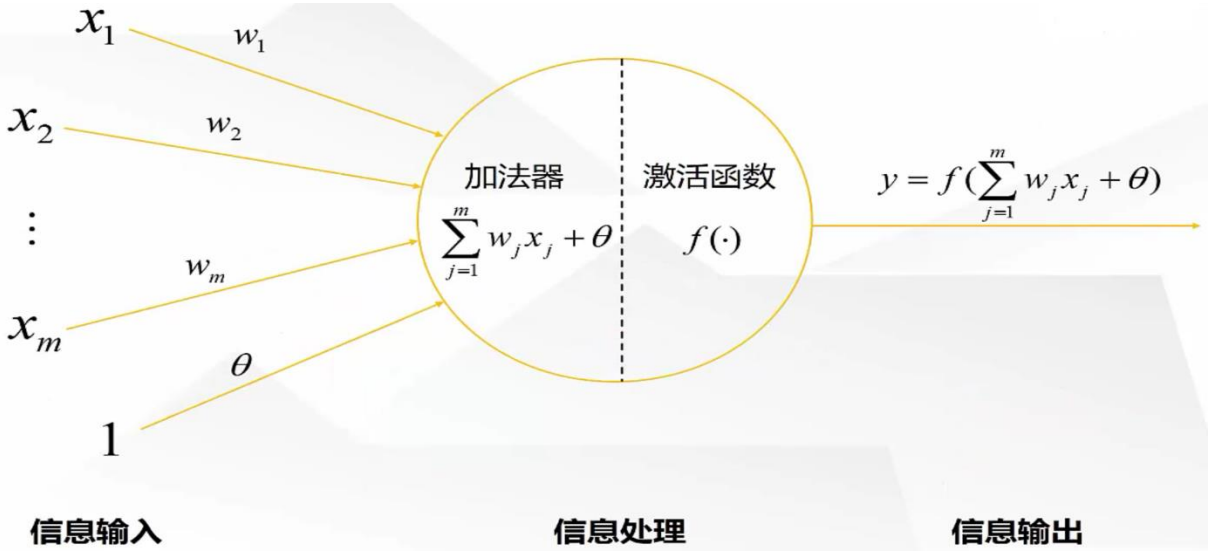


神经网络（*Neural Networks*，*NN*）

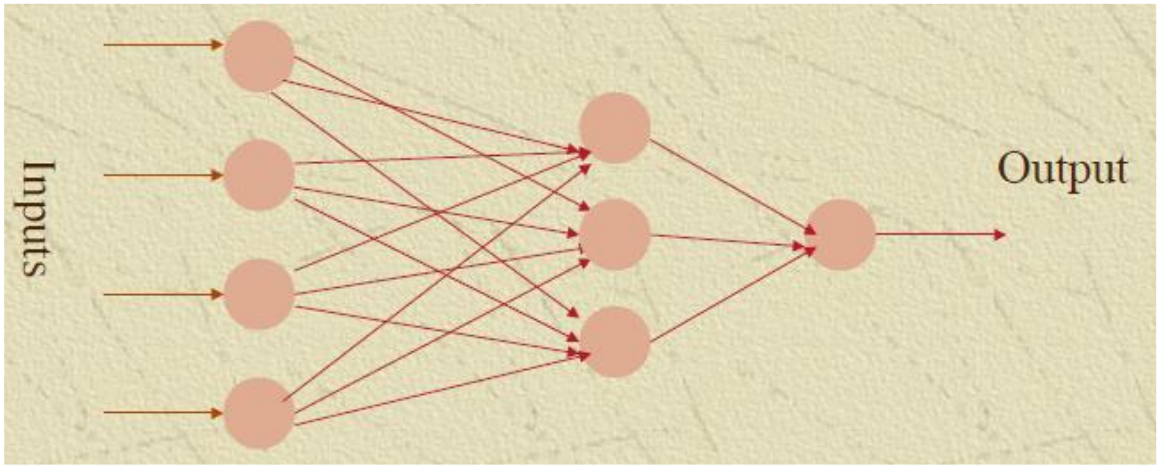
- 具有统计特性的数学模型。
- 创建思想源于人类神经网络的结构、功能和运行过程。
- 神经元（*Neurodes*）
  - 神经网络中的知识，表示为处理单元节点的集合。
- 突触（*Synapse*）
  - 每个节点与邻近层节点之间的加权连接。
- 权重（*Weight*）值
  - 连接间的权值，相当于神经网络的记忆。
- 阈值（*Threshold*）
  - 决定是否触发激励函数的生效的临界值
- 激励函数（*Activation Function*）
  - 每个节点的输出由一个输出函数计算所得。

神经元模型-从生物学结构到数学模型





人工神经网络

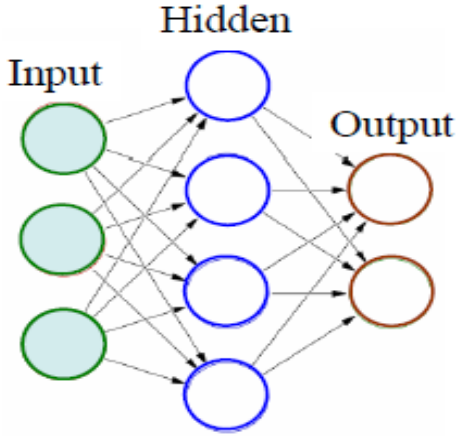


- 多个人工神经元按照特定的网络结构联接在一起，就构成了一个人工神经网络。
- 神经网络的目标就是将输入转换成有意义的输出。

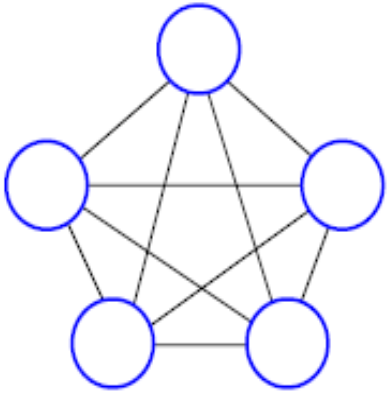


ANN常见结构

- 前馈结构 (***Feedforward Architecture***)
  - *without loops*
  - *static*

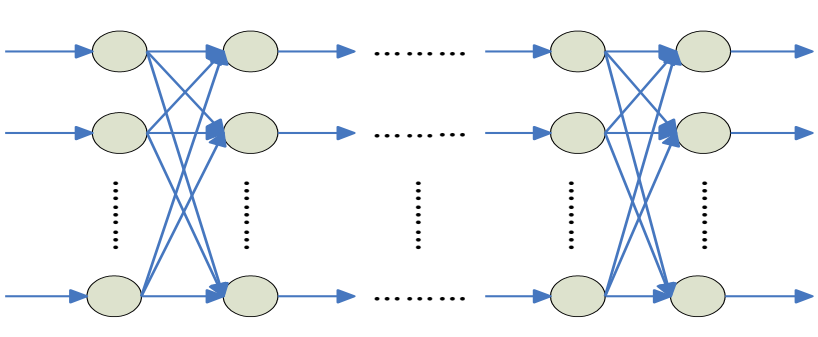


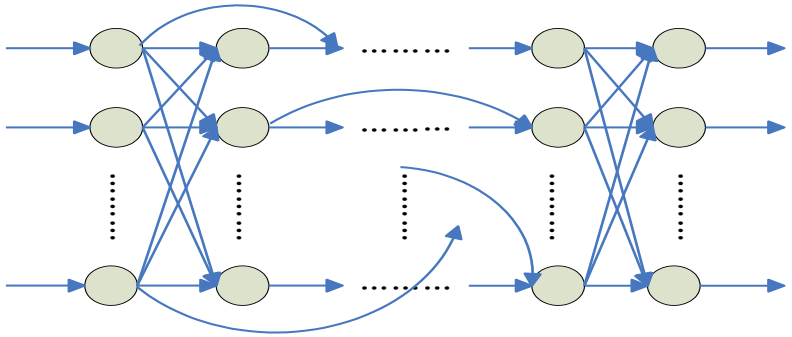
- 反馈/循环结构 (***Feedback/Recurrent Architecture***)
  - *with loops*
  - *dynamic (non-linear dynamical systems)*



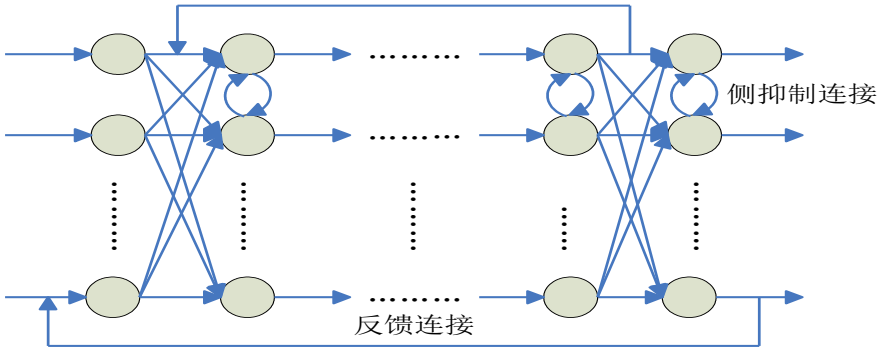


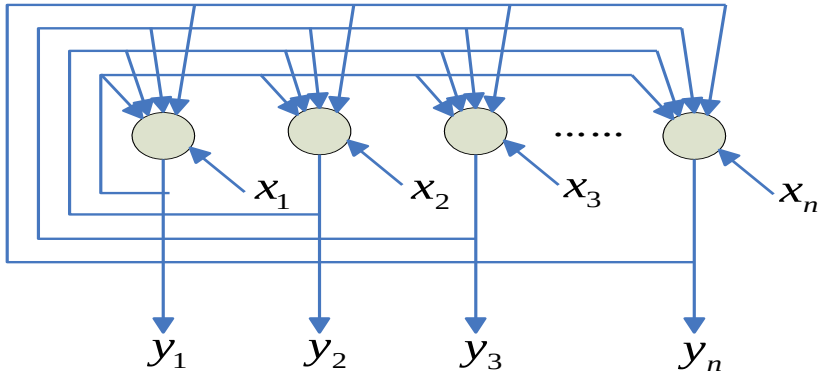
ANN常见结构





General structures of feedforward networks





General structures of feedback networks

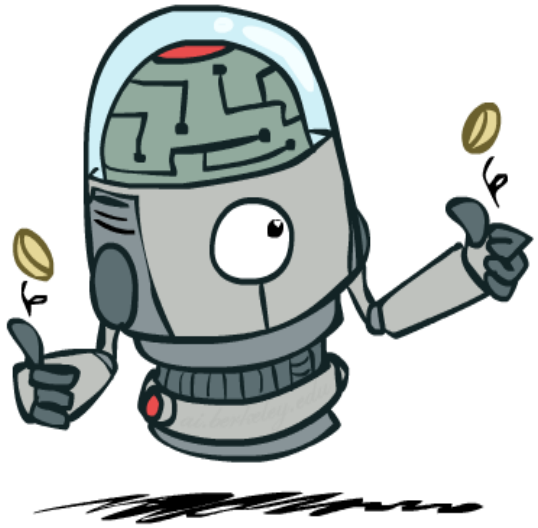


人工神经网络主要研究问题

- **两个主要问题：**
- **结构**

  如何让这些个体单元互相连接？

  How to interconnect individual units?



- **学习方法**

  如何自动确定ANN的连接权值乃至结构？

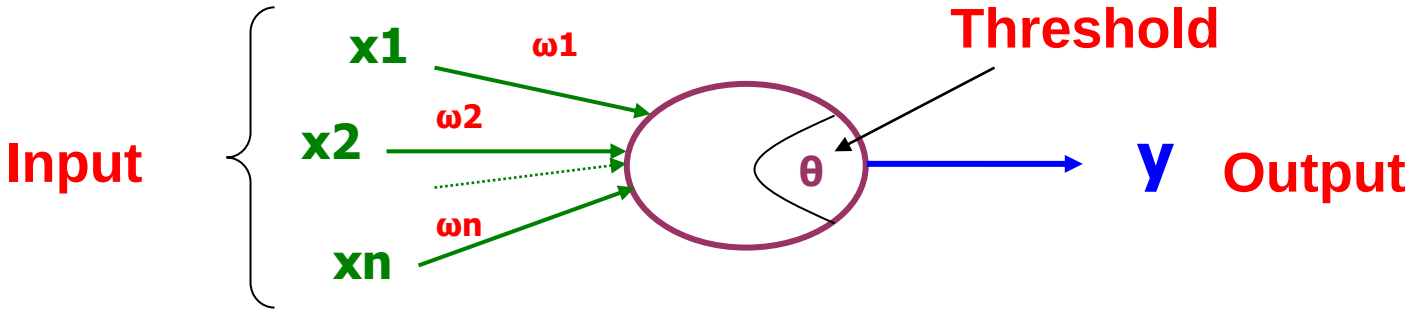
  How to automatically determine the connection weights or even structure of ANN?

Solutions to these two problems leads to a concrete ANN!



人工神经元模型

- M-P模型



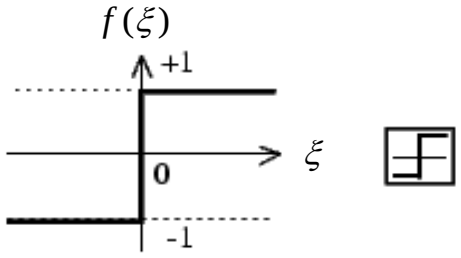
$$y = f(\xi) = f(g(X))$$

f: 激活函数(Activation Function)

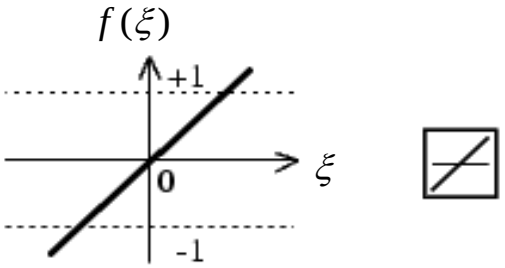
g: 组合函数(Combination Function)

McClloch and Pitts, *A logical calculus of the ideas immanent in nervous activity*, 1943

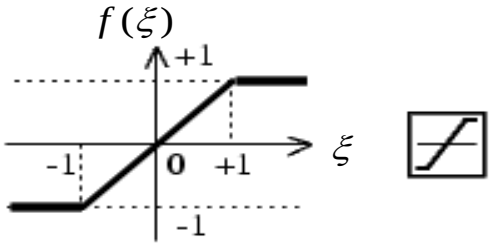
激励函数



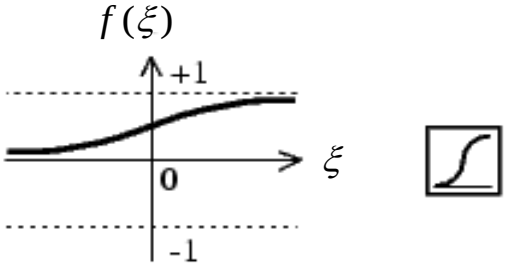
Threshold



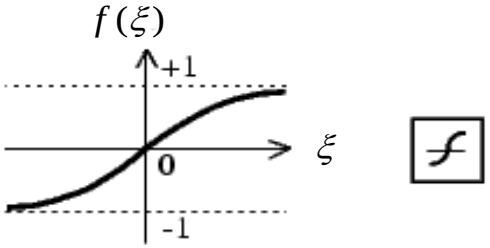
Linear



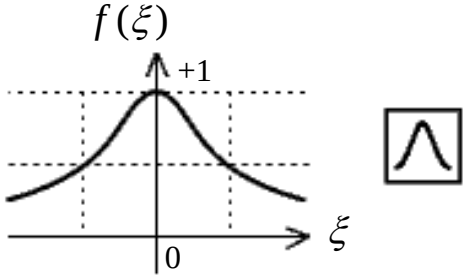
Saturating Linear



Logistic Sigmoid



Hyperbolic tangent Sigmoid



Gaussian

$$f(\xi)=\frac{1}{1+\exp(-\alpha\xi)}$$

$$f(\xi)=\tanh(\alpha\xi)$$



神经网络的输入和输出数据格式

1、神经网络输入格式

- 落在 **[0,1]** 闭区间内的数值类型
- 实际应用中，需要将分类数据变换为 **[0,1]** 区间的数值数据

两种方法：

- **方法一：**将 [0,1] 区间分为大小相等的间隔，将间隔点上的取值作为分类类型数据的数值表示
- **方法二：**对输入数据进行二进制编码，增加输入结点，用两个或多个输入节点表示一个输入属性（如：离散型属性）

2、神经网络输出格式

- 神经网络的输出结点表示为 **[0,1]** 区间内的连续值
- 如果神经网络是分类模型，需要对输出进行变换



神经网络模型

- **单层神经网络由一个输入层（Input layer）和一个输出层（Output layer）组成。**

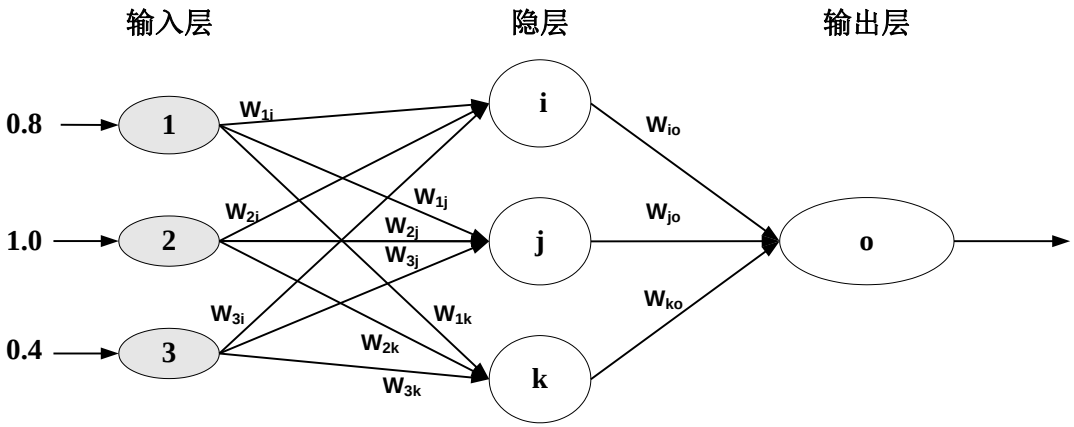


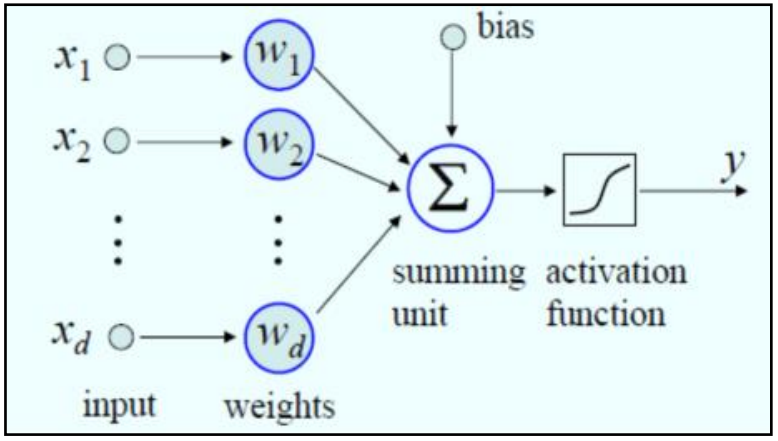
图6.1 全连接前馈神经网络结构

- **输入向量：输入数据，输入层节点数由训练实例的输入属性个数决定。**
- **输出向量：输出数据，输出层节点数由问题和应用决定。**
- **隐层个数和每个隐层内的节点个数一般可由用户指定，一般隐层被限定为两个，隐层节点数为输入节点的1.2倍到1.5倍。**



感知机（Perceptrons）

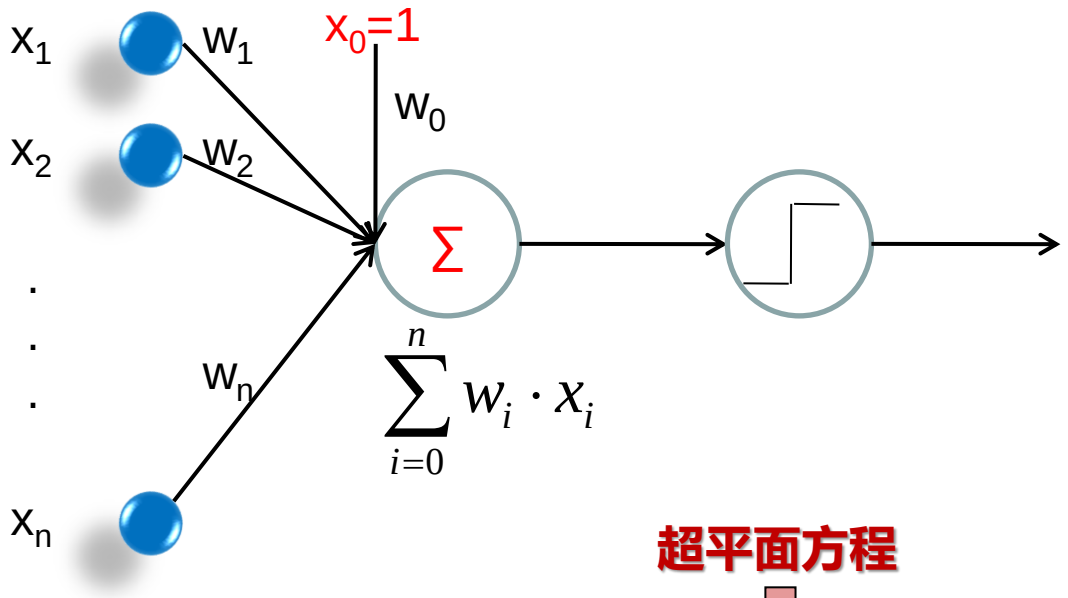
感知机（*Perceptrons*），***Frank Rosenblatt*** 在1957年就职于***Cornell*** 航空实验室(***Cornell Aeronautical Laboratory***)时所发明的一种人工神经网络。它可以被视为一种最简单形式的前馈式人工神经网络，是一种二元线性分类器 ***Frank Rosenblatt*** 给出了相应的感知器学习算法，常用的有感知机学习、最小二乘法 和 梯度下降法。譬如：感知机利用梯度下降法对损失函数进行极小化，求出可将训练数据进行线性划分的分离超平面，从而求得感知器模型。



深度学习：源于人工神经网络，含多隐层的多层感知机的深度学习结构。通过组合低层特征形成更加抽象的高层表示属性类别或特征，以发现数据的分布式特征表示。

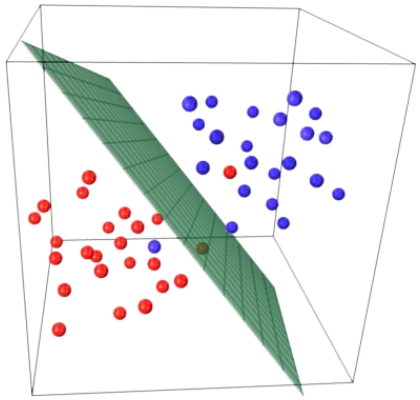
感知机

感知机以一个实数值向量作为输入，计算这些输入的线性组合，如果结果大于某个阈值，就输出 1，否则输出 0。

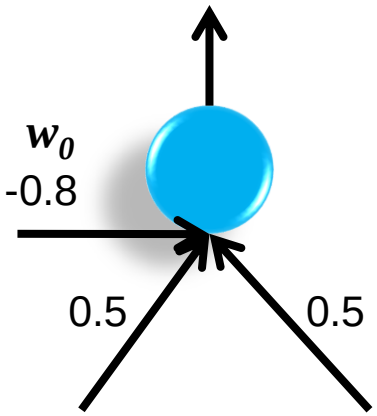


超平面方程

$$o(x_1,\ldots,x_n) = \begin{cases} 1, & if\ w_0 + w_1 \cdot x_1 + \cdots + w_n \cdot x_n > 0 \\ 0, & otherwise \end{cases}$$



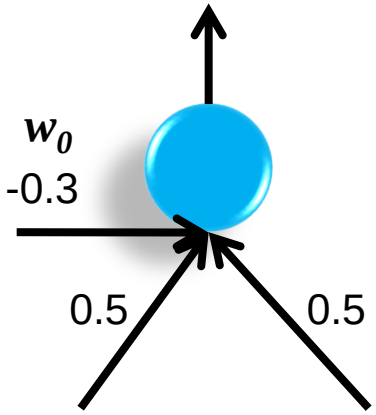
感知机的能力（Power of Perceptrons）



AND

$0.5x_1 + 0.5x_2 - 0.8 = 0$

超平面方程



OR

Input		∑	Output
0	0	-0.8	0
0	1	-0.3	0
1	0	-0.3	0
1	1	0.2	1

Input		∑	Output
0	0	-0.3	0
0	1	0.2	1
1	0	0.2	1
1	1	0.7	1

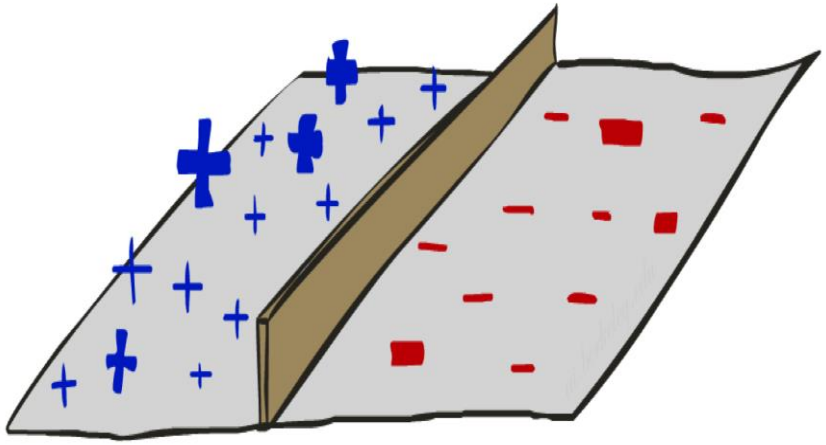
学习一个感知器意味着选择权值w_0, w_1, ……, w_n的值。所以感知机学习要考虑的：候选假设空间H 就是 所有可能的实数值权向量的集合。

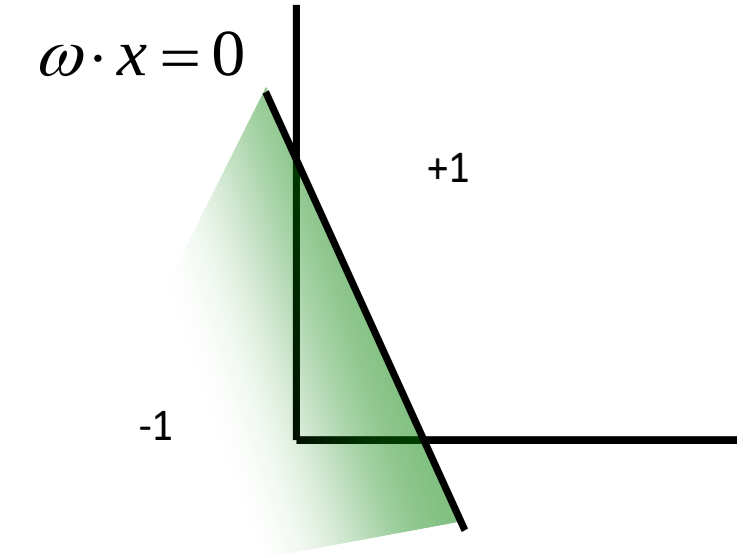
$$H = \{\vec{w} \mid \vec{w} \in R^{n+1}\}$$



感知机判别规则

- 输入空间中
  - 样本是空间中的一个点
  - 权向量是一个超平面
  - 超平面一边对应 $Y = 1$
  - 另一边对应 $Y = -1$
- 调整权值，减少训练集上的误差
- 简单的权值更新规则:
  1. 初始化
  2. 对每一个训练样本:
  3. *Classify with current weights*
  4. *If correct, no change!*
  5. *If wrong: adjust the weight vector*



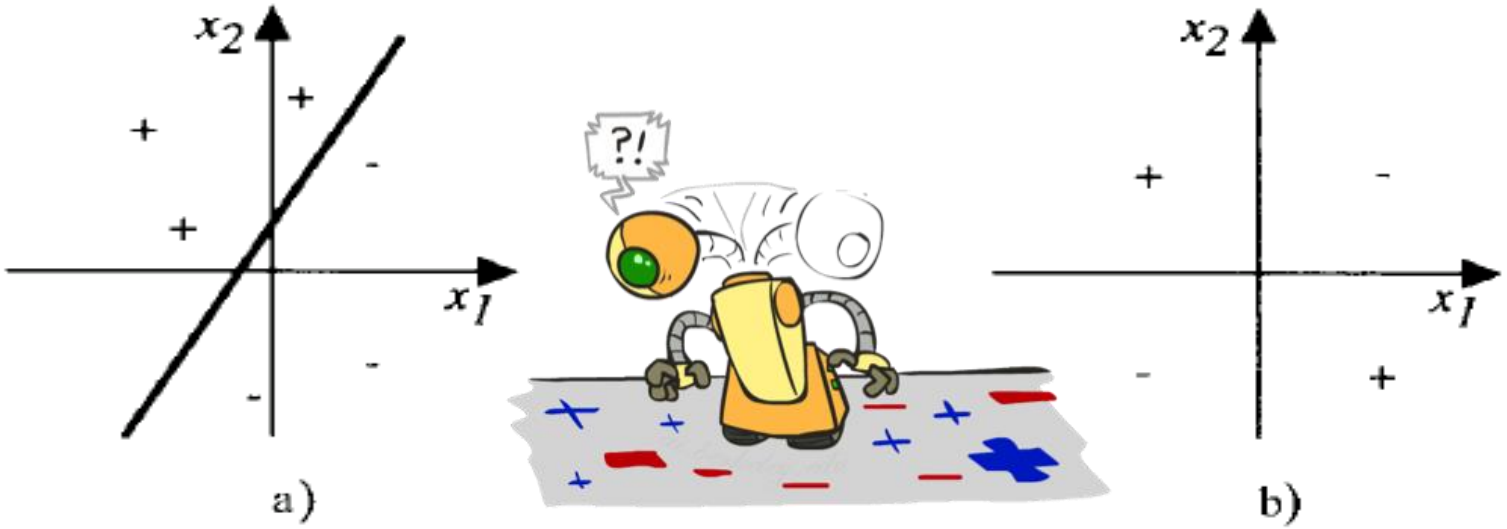




感知机的表征能力

- *AND* 和 *OR* 可被看作 *m- of-n* 函数的特例: 也就是要使函数输出为真，那么感知机的 *n* 个输入中至少 *m* 个必须为真:
  - *OR* 函数对应于*m = 1*
  - *AND* 函数对应于*m = n*
- 任意 *m-of-n* 函数可以很容易地用感知机表示，只要设置所有输入的权为同样的值(如0.5)，然后据此恰当地设置阈值。
- 感知机可以表示所有的原子布尔函数:与、或、与非和或非。
- 然而，一些布尔函数无法用单一的感知机表示，例如异或函数(*XOR*)，它当且仅当 *x1* ≠ *x2* 时输出为1。

感知器的表征能力



两输入感知器表示的决策面

a)一组训练样例和一个能正确分类这些样例的感知机决策面

b)一组非线性可分的训练样例(也就是不能用任一直线正确分类的样例)。x_1和x_2是感知器的输入。“+”表示正例，“-”表示反例。



感知机训练法则

- 感知机的学习任务是决定一个**权向量**，它可以使感知机对于给定的训练样例输出正确的1或0(拟合训练数据)
- 两种算法:**感知机法则**和 *delta*法则。这两种算法保证收敛到**可接受的假设**，在不同的条件下收敛到的假设略有不同。这两种方法提供了学习多个单元构成的网络的基础。



感知机训练法则

为了得到可接受的权向量，有2种学习方法

一种办法是-依次学习：

- **从随机的权值开始**
- **然后反复地应用这个感知机到每个训练样例**
- **只要它误分类样例就修改感知器的权值**
- **重复这个过程，直到感知机正确分类所有的训练样例**

每一步根据感知器训练法则来修改权值：

$$w_{i+1} \leftarrow w_i + \Delta w_i$$

另一种办法是-批学习：

每训练一组样本通过误差之和来调整修改权值



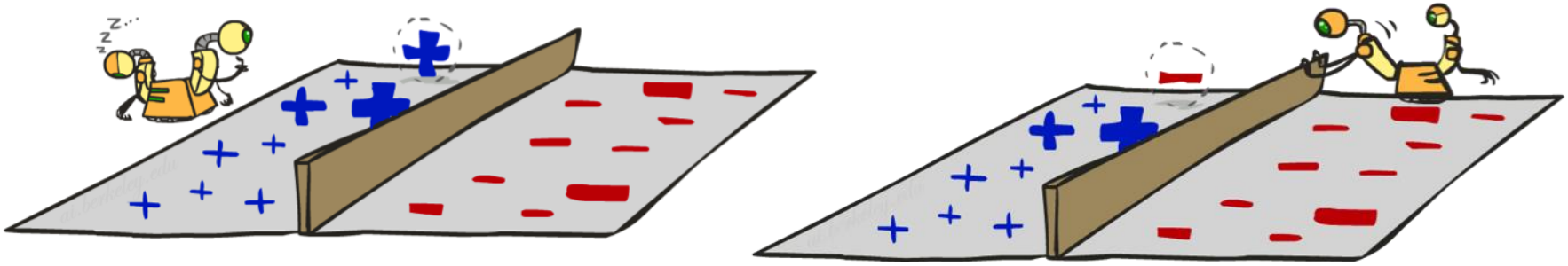
感知机训练法则

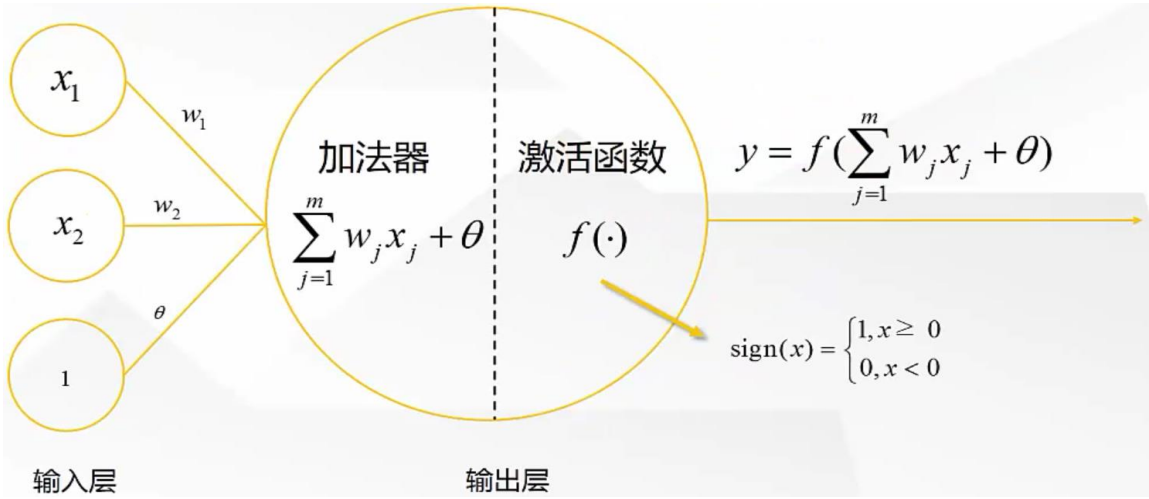
可以证明：

在有限次地使用感知机训练法则后，上面的训练过程会收敛到一个能正确分类所有训练样例的权向量，前提是训练样例线性可分，并且使用了充分小的η。

如果数据不是线性可分的，那么不能保证训练过程收敛。

感知机的学习算法（训练）





感知机需要求什么确定什么？

x_1，x_2，x_3... x_j 代表什么？

输入层为什么有1权值为θ？

权值如何更新？

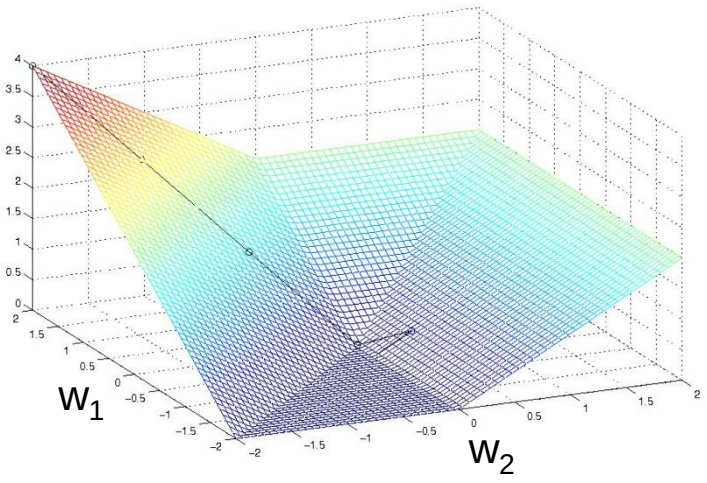
$$E = \sum_{j=1}^{m} \frac{1}{2}(y - \hat{y})^2$$

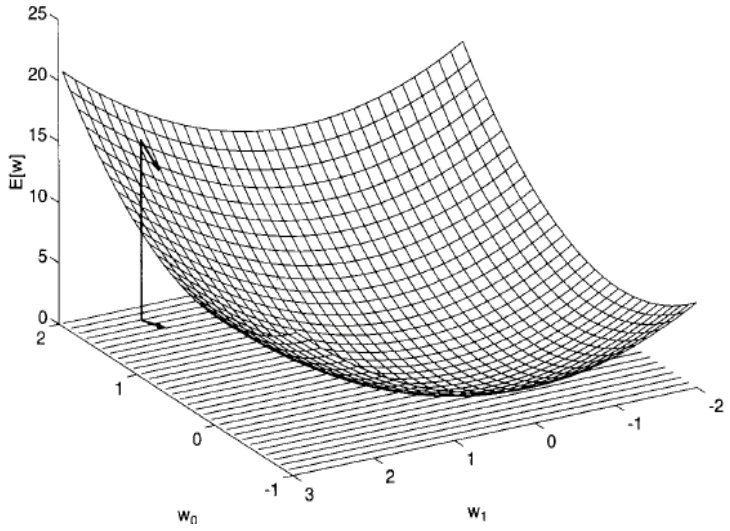
$$w_{j+1} = w_j + \Delta w_j$$

$$\Delta w_j = \eta (y - \hat{y}) x_j$$

j 不是样本，是样本的第 *j* 个属性

误差曲面（*Error Surface*）





✓按照 ***E*** 的定义，对于线性单元，这个误差曲面必然是具有单一全局最小值的抛物面。

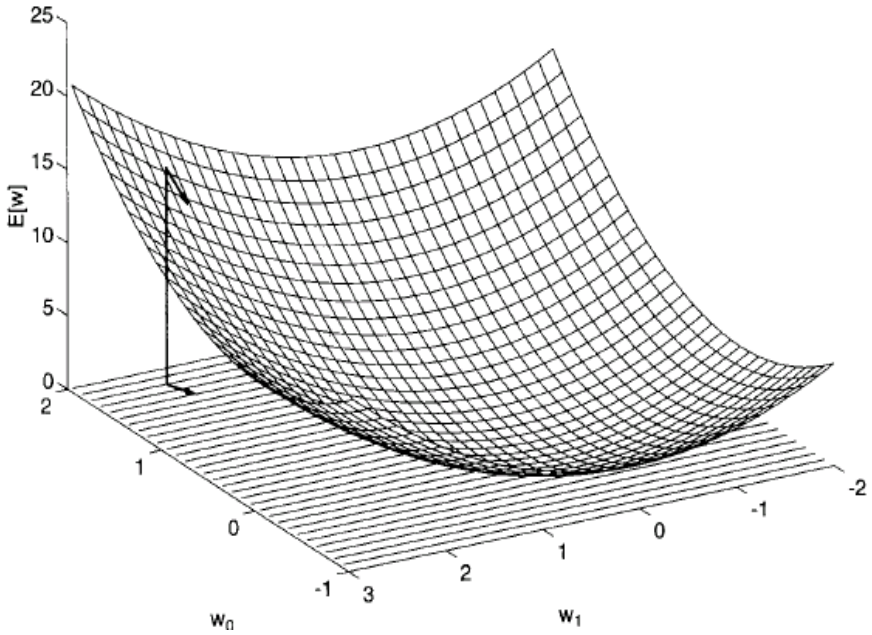
✓具体的抛物面形状依赖于具体的训练样例集合。

误差曲面（*Error Surface*）

确定一个使 *E* 最小化的权向量:

- **梯度下降搜索从一个任意的初始权向量开始**
- **然后以很小的步伐反复修改这个向量:**

  每一步都沿误差曲面产生最陡峭下降的方向修改权向量
- **继续这个过程,直到得到全局的最小误差点。**





什么叫梯度下降?

1.单变量的函数:

$$J(\theta) = \theta^2$$

2.函数的微分:

$$J'(\theta) = 2\theta.$$

3.初始化起点:

$$\theta^0 = 1$$

4.学习率:

$$\alpha = 0.4$$

5.根据梯度下降的计算公式:

$$\Theta^1 = \Theta^0 - \alpha \nabla J(\Theta) \quad \text{evaluated at } \Theta^0$$

6.我们开始进行梯度下降的迭代计算过程:

$$\theta^0 = 1$$

$$\theta^1 = \theta^0 - \alpha * J'(\theta^0)$$

$$= 1 - 0.4 * 2$$

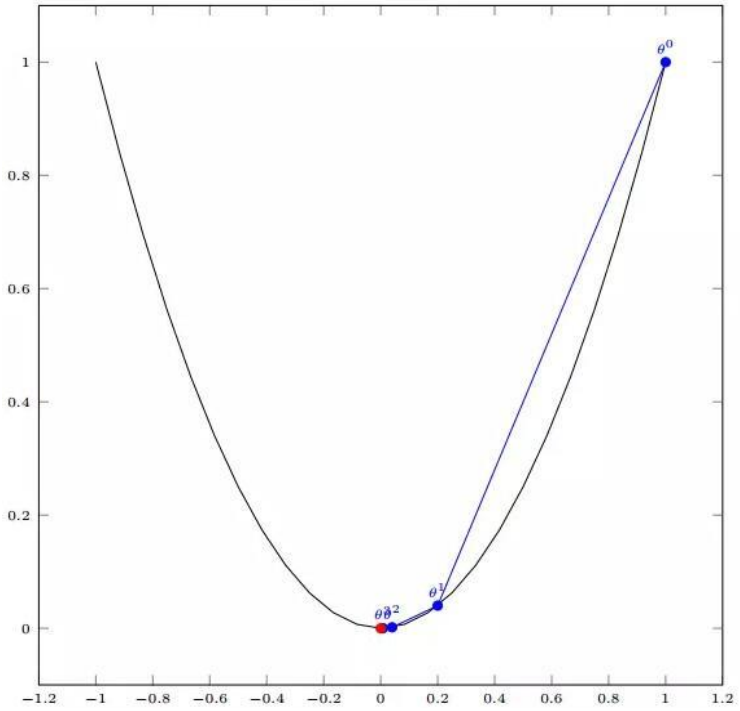
$$= 0.2$$

$$\theta^2 = \theta^1 - \alpha * J'(\theta^1)$$

$$= 0.04$$

$$\theta^3 = 0.008$$

$$\theta^4 = 0.0016$$



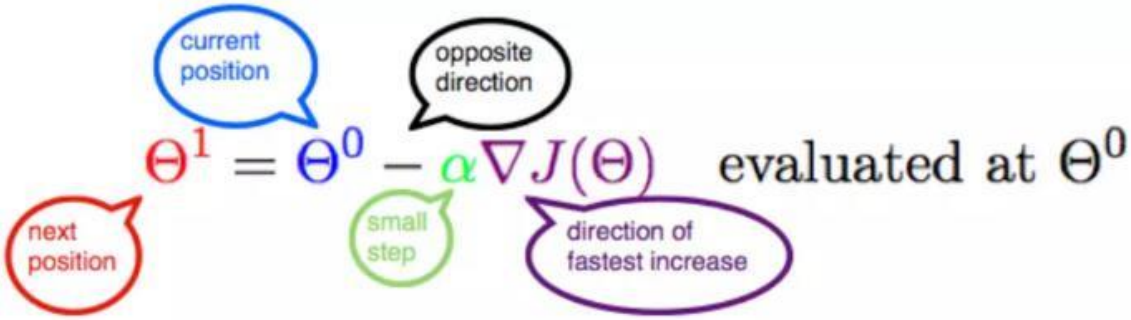
如图，经过四次的迭代，也就是走了四步，基本就抵达了函数的最低点，也就是山底





什么是学习率?

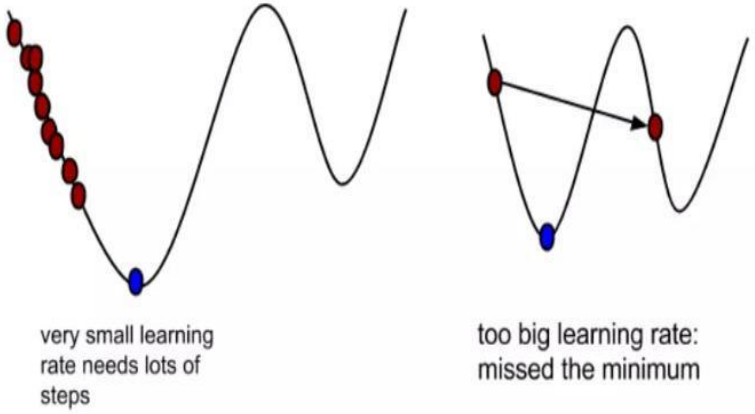
梯度下降公式:



α 在梯度下降算法中被称作为学习率或者步长;

意味着:通过 α 来控制每一步走的距离,以保证不要步子跨的太大,也就是不要走太快,错过了最低点。同时也不能走的太慢,导致太阳下山了,还没有走到山下。

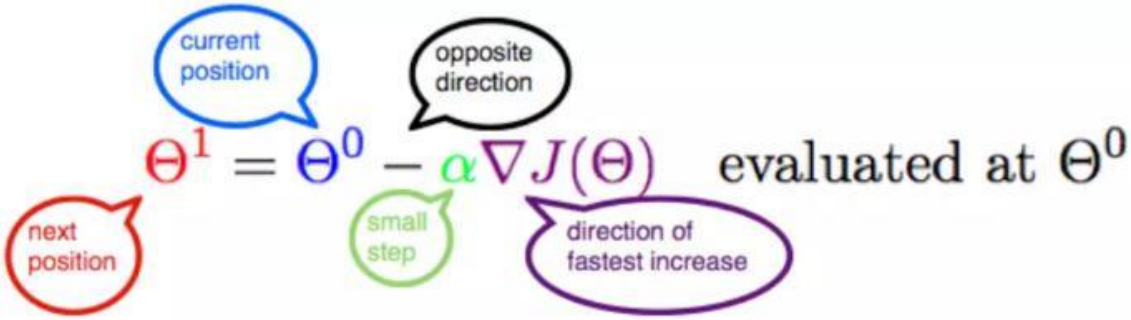
所以:α 在梯度下降中往往是很重要的!α 不能太大也不能太小,太小的话,可能导致迟迟走不到最低点,太大的话,会导致错过最低点!





什么是学习率?

梯度下降公式:



为什么要梯度前面是一个负号?

梯度前加一个负号，就意味着朝着梯度相反的方向前进!

梯度的方向实际就是函数在此点上升最快的方向！而我们需要朝着下降最快的方向走，自然就是负的梯度的方向，所以此处需要加上负号



梯度下降和*delta*法则（1/2）

- **问题提出：如果训练样本不是线性可分的，那么感知器训练法则不能保证训练过程收敛，*delta*法则会收敛到目标概念的最佳近似。**
- ***delta*法则的关键思想是使用梯度下降来搜索可能的权向量的假设空间，以找到最佳拟合训练样例的权向量。**
- ***delta*法则为反向传播算法（*BP*）提供了基础。且对于包含多种不同类型的连续参数化假设的假设空间，梯度下降是必须遍历这样的假设空间的所有学习算法的基础。**



梯度下降和*delta*法则（2/2）

把delta训练法则理解为训练一个无阈值的感知器，也就是一个线性单元，它的输出 o 如下：

$$o(\vec{x}) = \vec{w} \cdot \vec{x}$$

为了推导线性单元的权值学习法则，先指定一个度量标准来衡量假设相对于训练样例的训练误差。一个常用的特别方便的度量标准为：

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

在一定的条件下，对于给定的全部训练数据使 *E* 最小化的假设也就是 *H* 中最可能的假设



梯度下降（*Gradient Descent*）

$$E(\vec{w}) \equiv \frac{1}{2}\sum_{d \in D}(t_d - o_d)^2$$

批学习（Batch Learning）

可以通过计算 E 相对向量 $\vec{w}$ 的每个分量的导数来得到沿误差曲面最陡峭下降的这个方向。这个向量导数被称为 E 对于 的梯度 (*Gradient*)，记作：

$$\nabla E(\vec{w}) \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_n}\right]$$

E 分别对 w 求偏导

为什么是偏导?

既然梯度确定了E 最陡峭上升的方向，那么梯度下降的训练法则是:



$$w_i \leftarrow w_i + \Delta w_i \quad where \quad \Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

学习率（Learning Rate）



Delta法则（*Delta Rule*）

可以从公式中计算 E 的微分，从而得到组成这个梯度向量的分量 $\frac{\partial E}{\partial w_i}$

过程如下：

$$\frac{\partial E}{\partial w_i} = \frac{\partial}{\partial w_i}\frac{1}{2}\sum_{d\in D}(t_d - o_d)^2$$

$$= \frac{1}{2}\sum_{d\in D}\frac{\partial}{\partial w_i}(t_d - o_d)^2$$

$$= \frac{1}{2}\sum_{d\in D}2(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - o_d)$$

$$= \sum_{d\in D}(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - w\cdot x_d)$$

$$= \sum_{d\in D}(t_d - o_d)(-x_{id})$$

$$\Delta w_i = -\eta\frac{\partial E}{\partial w_i}$$

$$\Delta w_i = \eta\sum_{d\in D}(t_d - o_d)x_{id}$$

$$o(x) = w\cdot x$$





训练线性单元的梯度下降算法（标准梯度下降）

GRADIENT-DESCENT(training-examples,η)

*training-examples*中每一个训练样例形式为序列<*x*, *t*>，其中x是输入值向量，*t*是目标输出值，η是学习速率(例如0.05)

初始化每个 w_i 为某个小的随机值

For 遇到终止条件之前，做以下操作:　　批学习（Batch Learning）

初始化每个 Δw_i 为0

对于训练样例training-examples中的 每个<*x*, *t*> 做:

把实例x输人到此单元，计算输出o

对于线性单元的每个权 w_i 做

$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i$

For 对于线性单元的每个权 w_i 做：　$w_i \leftarrow w_i + \Delta w_i$

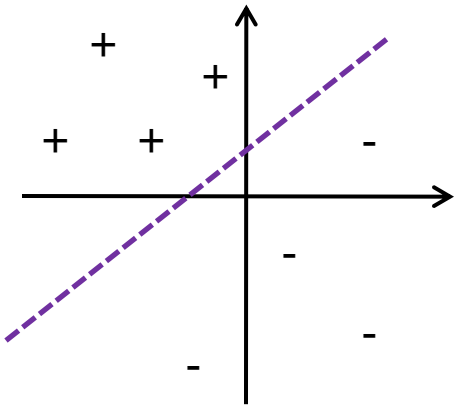


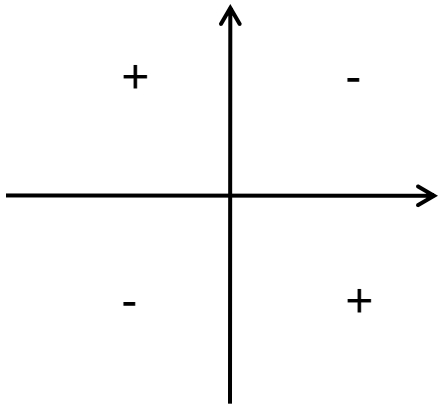
依次随机学习（*Stochastic Learning*）

$$w_i \leftarrow w_i + \Delta w_i \qquad where \quad \Delta w_i = \eta(t-o)x_i$$

For example, if x_i=0.8, η=0.1, t=1 and o=0

$\Delta w_i = \eta(t-o)x_i$ = 0.1✖(1-0) ✖ 0.8 = 0.08







依次随机学习（*Stochastic Learning*）：与非门

Input			Target	Initial Weights			Output					Error	Correction	Final Weights		
							Individual			Sum	Final Output					
x_0	x_1	x_2	t	w_0	w_1	w_2	C_0	C_1	C_2	S	o	E	R	w_0	w_1	w_2
							$x_0 \cdot w_0$	$x_1 \cdot w_1$	$x_2 \cdot w_2$	$C_0+C_1+C_2$		t-o	LR x E			
1	0	0	1	0	0	0	0	0	0	0	0	1	+0.1	0.1	0	0
1	0	1	1	0.1	0	0	0.1	0	0	0.1	0	1	+0.1	0.2	0	0.1
1	1	0	1	0.2	0	0.1	0.2	0	0	0.2	0	1	+0.1	0.3	0.1	0.1
1	1	1	0	0.3	0.1	0.1	0.3	0.1	0.1	0.5	0	0	0	0.3	0.1	0.1
1	0	0	1	0.3	0.1	0.1	0.3	0	0	0.3	0	1	+0.1	0.4	0.1	0.1
1	0	1	1	0.4	0.1	0.1	0.4	0	0.1	0.5	0	1	+0.1	0.5	0.1	0.2
1	1	0	1	0.5	0.1	0.2	0.5	0.1	0	0.6	1	0	0	0.5	0.1	0.2
1	1	1	0	0.5	0.1	0.2	0.5	0.1	0.2	0.8	1	-1	-0.1	0.4	0	0.1
1	0	0	1	0.4	0	0.1	0.4	0	0	0.4	0	1	+0.1	0.5	0	0.1
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
1	1	0	1	0.8	-.2	-.1	0.8	-.2	0	0.6	1	0	0	0.8	-.2	-.1

Threshold=0.5　　Learning Rate=0.1

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d) x_{id}$$



单层感知机

– If the activation function is a linear function

$$y_k(\mathbf{x}) = v_k = \sum_{i=1}^{d} w_{k,i} x_i + w_{k,0} = \mathbf{w}_k^T \tilde{\mathbf{x}},\ k = 1, 2, \ldots, m$$

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^T \quad (\mathbf{T} \approx \mathbf{X}\mathbf{W}^T)$$

$$\min_{\mathbf{W}} E = \min_{\mathbf{W}} \frac{1}{2} \sum_{i=1}^{n} \sum_{k=1}^{m} [y_k(\mathbf{x}_i) - t_{k,i}]^2 = \min_{\mathbf{W}} \frac{1}{2} \| \mathbf{X}\mathbf{W}^T - \mathbf{T} \|_F^2$$

$$\frac{\partial E}{\partial \mathbf{W}} = 0 \Rightarrow \mathbf{W}^T = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$$

完全等价于线性回归！



单层感知机

– If the activation function is a sigmoid function

• $\mathbf{W}_{m\times(d+1)}$ can be updated by gradient descent method

$$E=\frac{1}{2}\sum_{i=1}^{n}\sum_{k=1}^{m}[y_k(\mathbf{x}_i)-t_{k,i}]^2$$

$$\mathbf{w}_k(t+1)=\mathbf{w}_k(t)-\eta_t\frac{\partial E}{\partial \mathbf{w}_k}$$

齐次坐标

$$\frac{\partial E}{\partial \mathbf{W}_k}=\sum_{i=1}^{n}\sum_{k=1}^{m}[y_k(\mathbf{x}_i)-t_{k,i}]\cdot y_k(\mathbf{x}_i)\cdot[1-y_k(\mathbf{x}_i)]\cdot\mathbf{x}_i,\quad y_k(\mathbf{x})=g(v_k)=\frac{1}{1+e^{-\mathbf{w}_k^T\tilde{\mathbf{x}}}}$$



单层感知机

- Stochastic approximation （每次考虑一个训练样本）

$$E_i = \frac{1}{2}\sum_{k=1}^{m}[y_k(\mathbf{x}_i) - t_{k,i}]^2$$

Sequential updating

$$\mathbf{w}_k(t+1) = \mathbf{w}_k(t) - \eta_t \frac{\partial E_i}{\partial \mathbf{w}_k}$$

第i个样本

齐次坐标表示

$$\frac{\partial E_i}{\partial \mathbf{w}_k} = \sum_{k=1}^{m}[y_k(\mathbf{x}_i) - t_{k,i}] \cdot y_k(\mathbf{x}_i) \cdot [1 - y_k(\mathbf{x}_i)] \cdot \tilde{\mathbf{x}}_i$$

Lemma: The stochastic approximation converges to a local minimum of E with probability 1 under the following conditions:

$$\lim_{t\to\infty} \eta(t) = 0, \quad \sum_{t=1}^{\infty} \eta(t)^2 < \infty$$

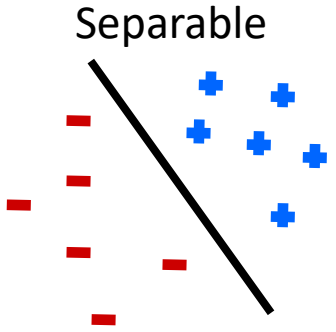
感知机特性

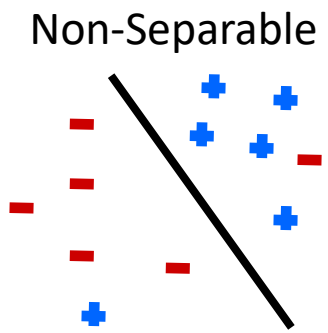
- **线性可分性:**

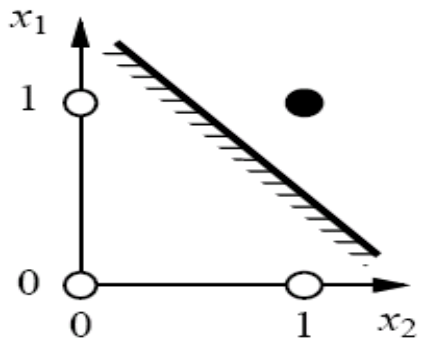
  确实可以通过训练集使权参数能够把数据分类， 可以处理AND，OR，NOT，…等问题，但无法处理 XOR

- **算法收敛性:**

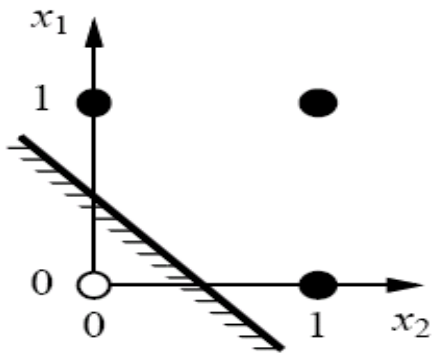
  如果训练是可分离的，感知机最终会收敛(二分类情况)



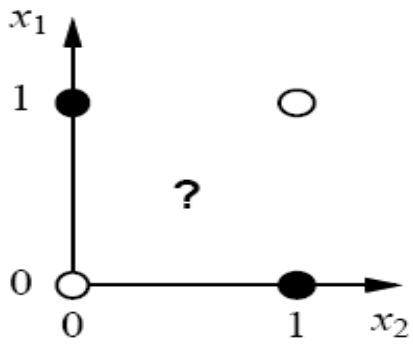




(a) x_1 and x_2



(b) x_1 or x_2



(c) x_1 xor x_2



感知机存在的问题

- **噪声（不可分情况）：**

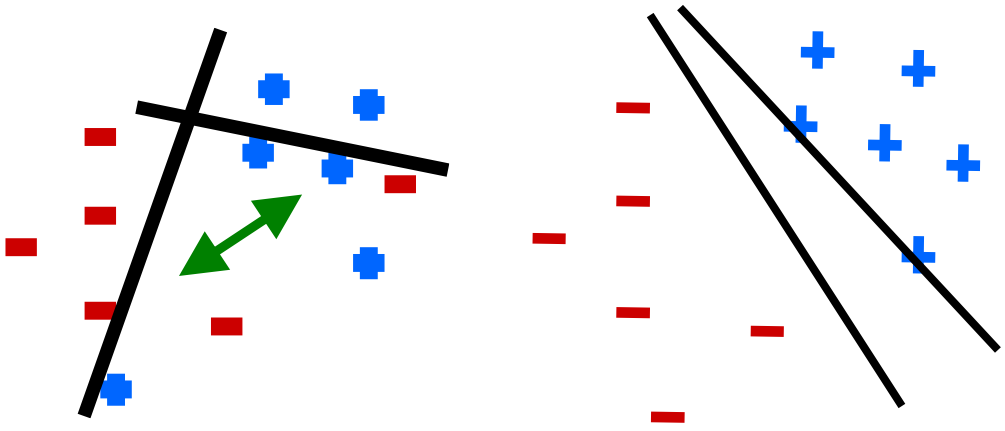
  如果数据不可分离，权重可能会剧烈波动

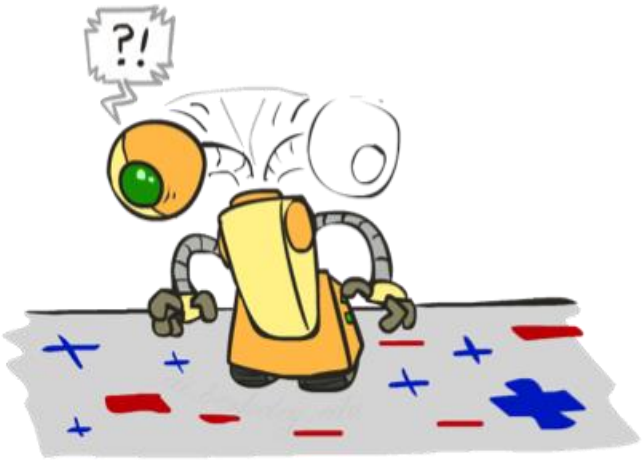
- **泛化性:**

  找到一个“勉强”分离的解决方案

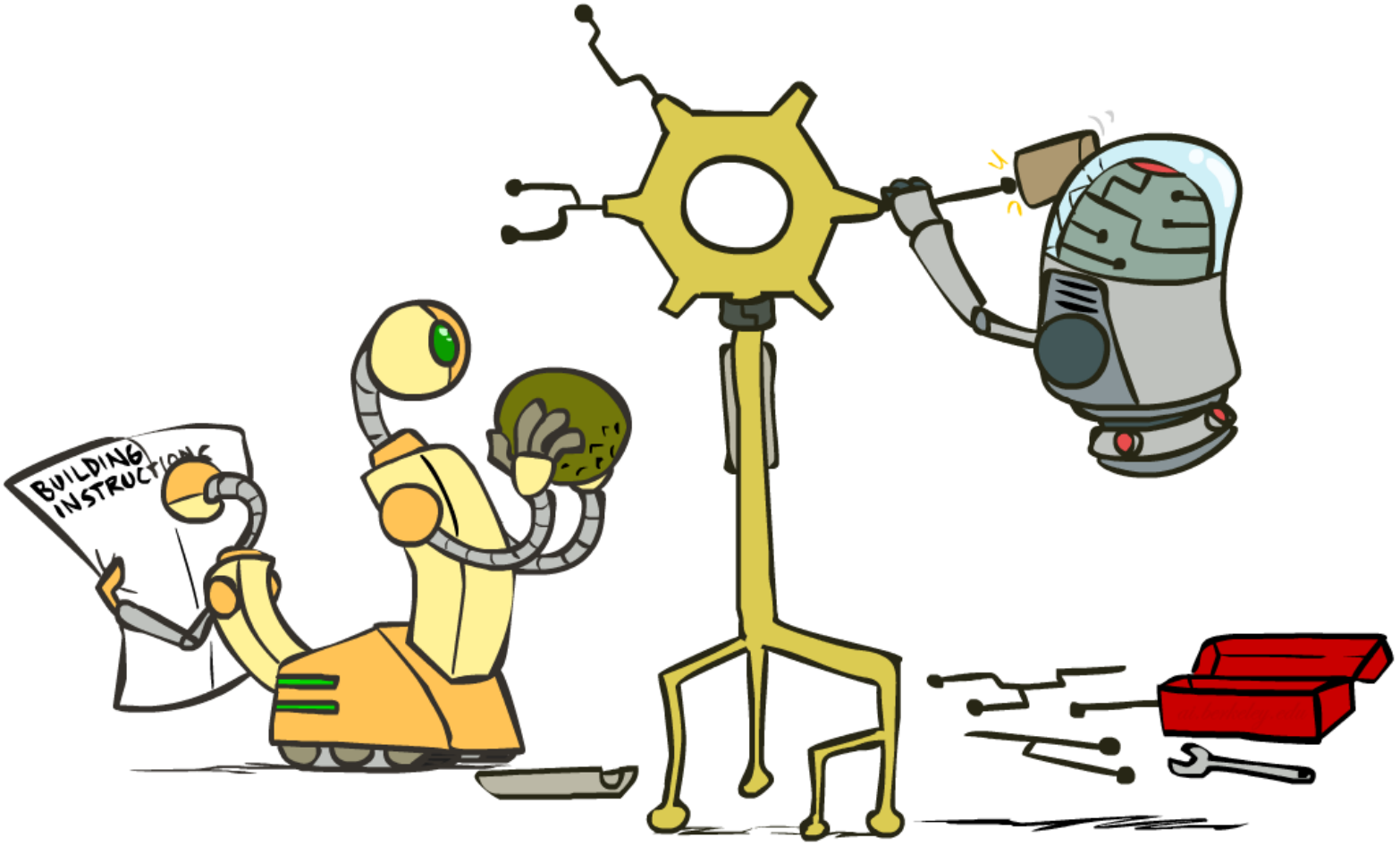
- **无法解决线性不可分问题**





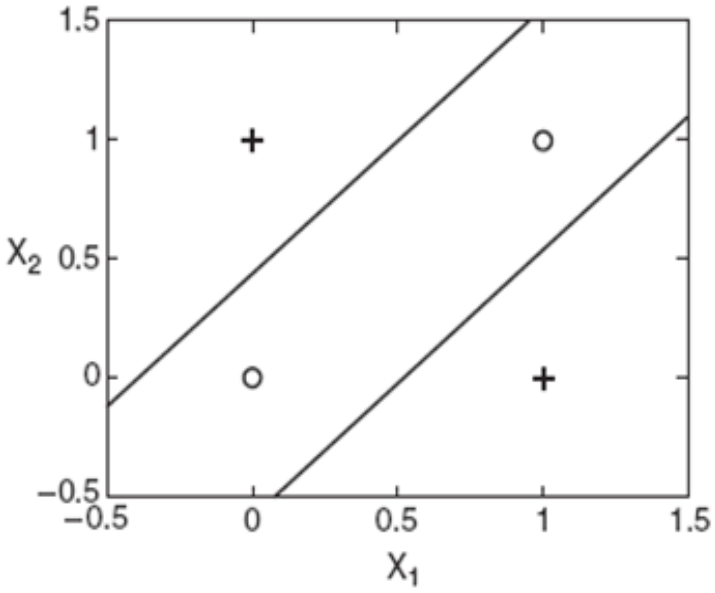


改进感知机

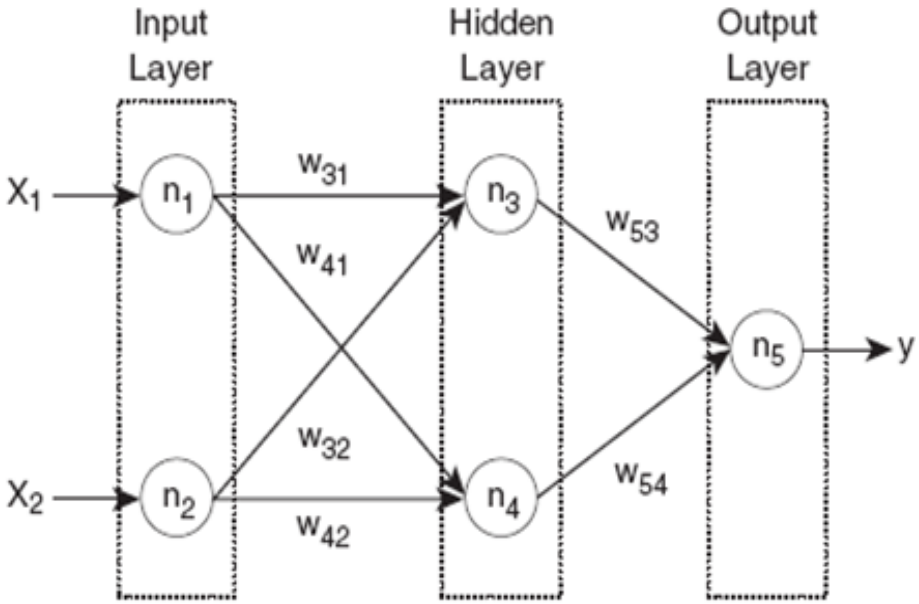


感知机改进-多层感知机（*Multilayer Perceptron*）

- **例：用两层前馈神经网络解决异或问题**



(a) 决策边界

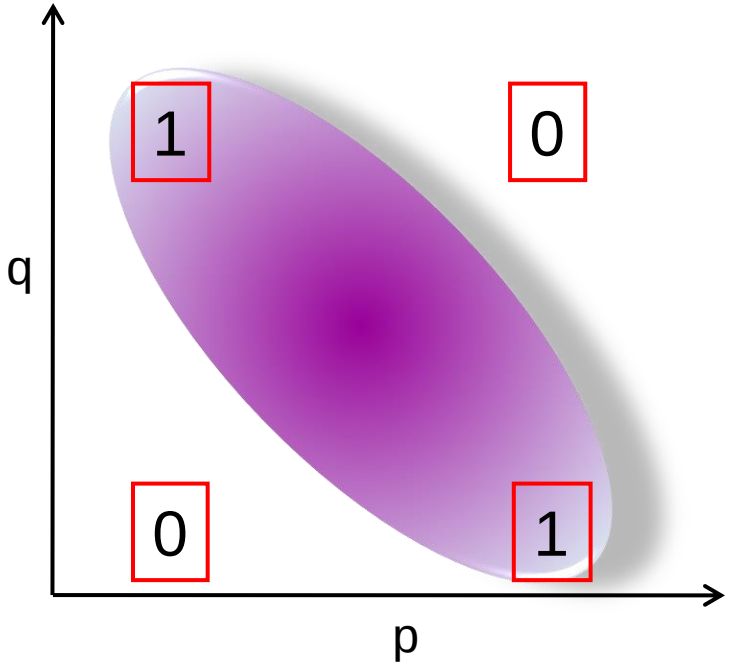


(b) 神经网络拓扑结构



XOR

$$p \oplus q = p\bar{q} + \bar{p}q = (p+q)(\overline{pq})$$



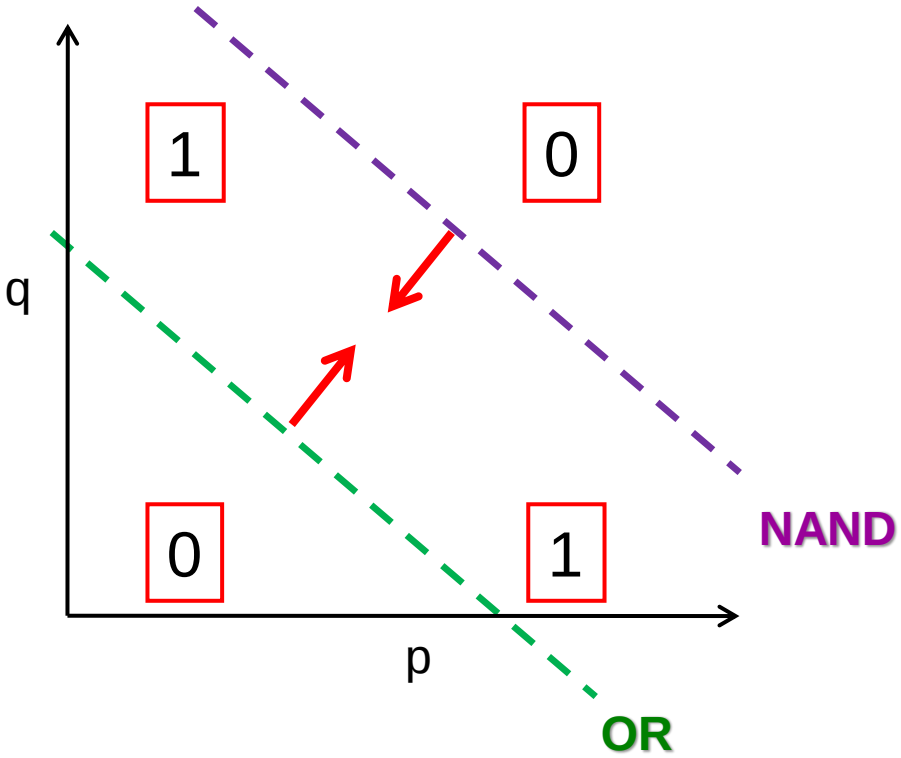
Input		Output
0	0	0
0	1	1
1	0	1
1	1	0

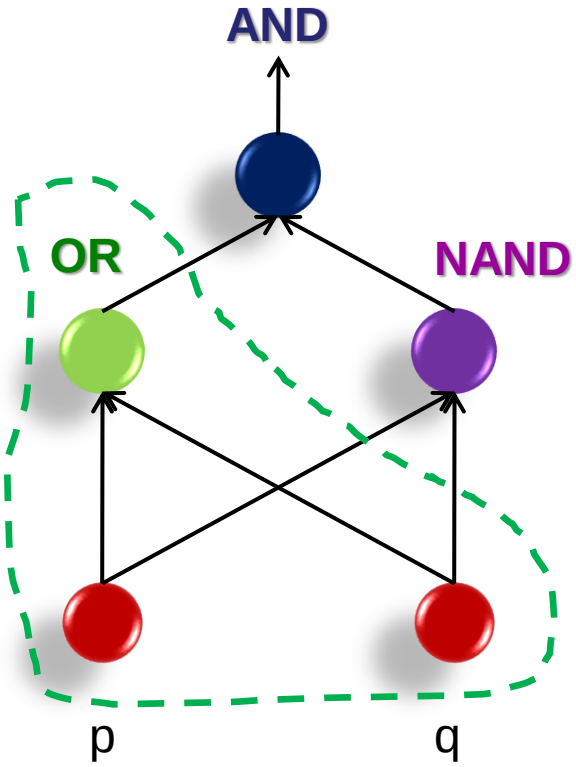
Cannot be separated by a single line.



XOR

$$p \oplus q = \neg(p \wedge q) \wedge (p \vee q)$$





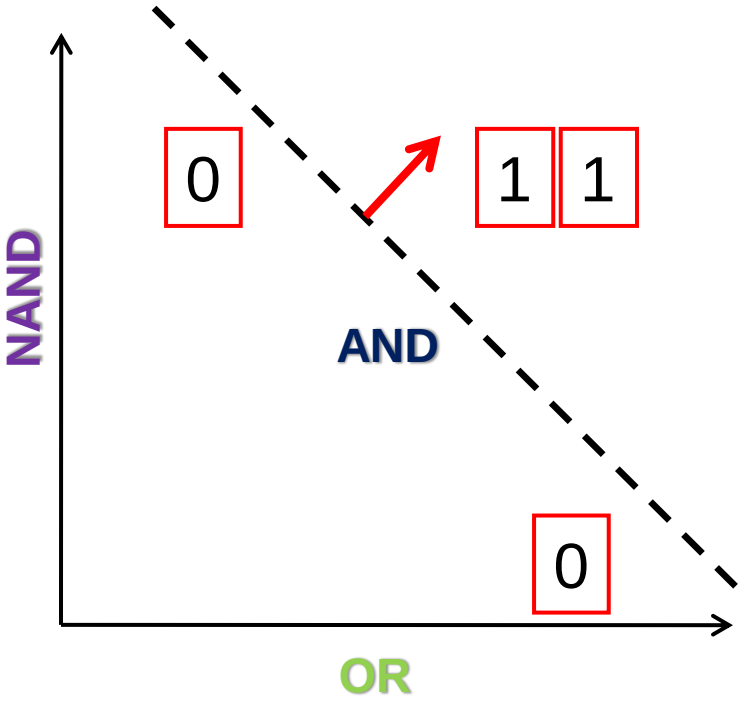


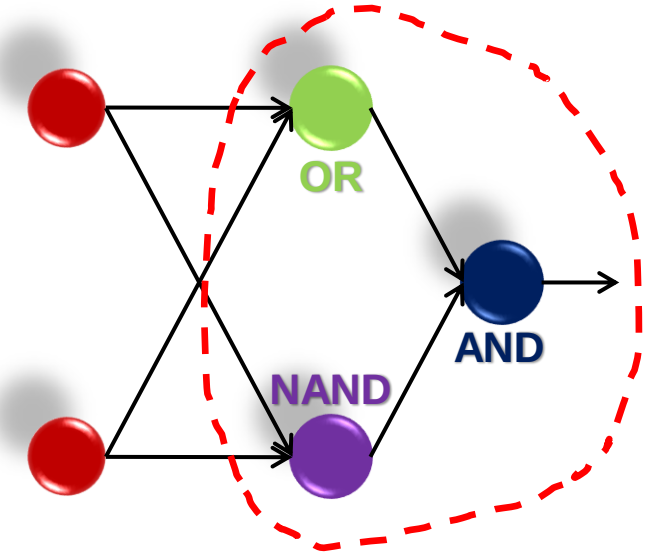
Hidden Layer Representations

p	q	OR	NAND	AND
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

Input　Hidden　Output

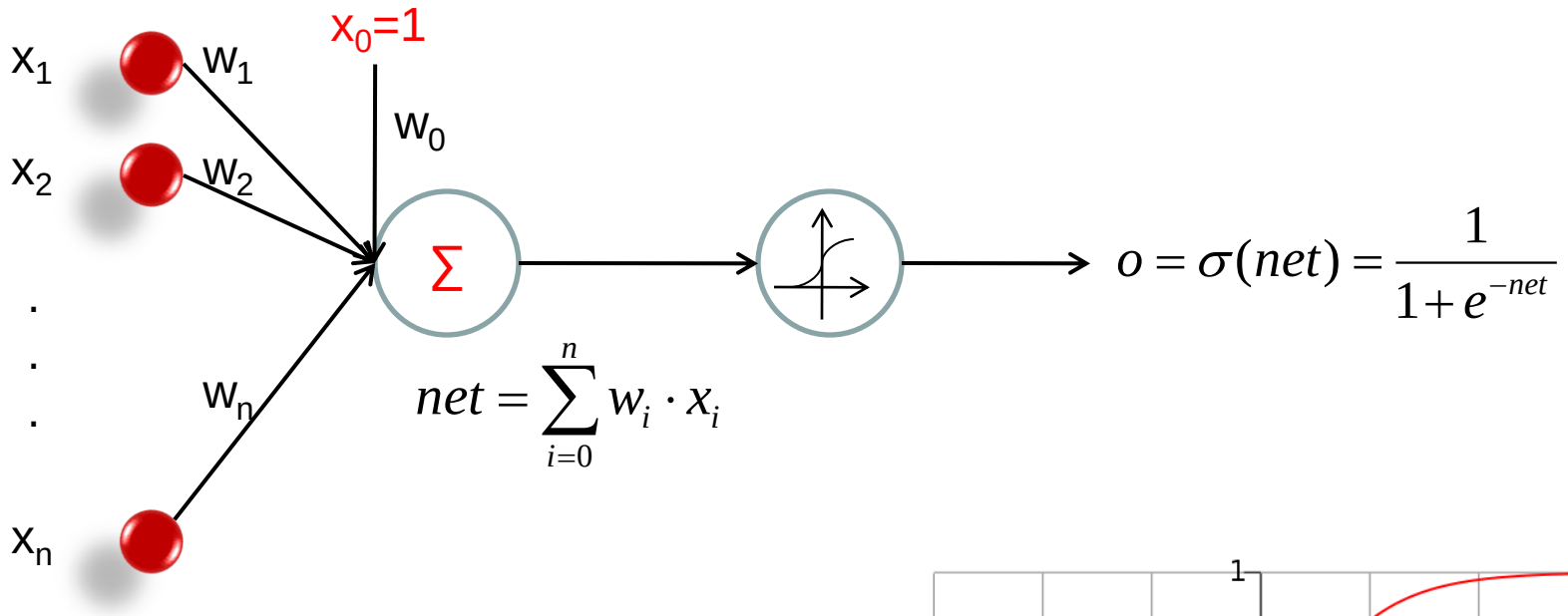
Mapping







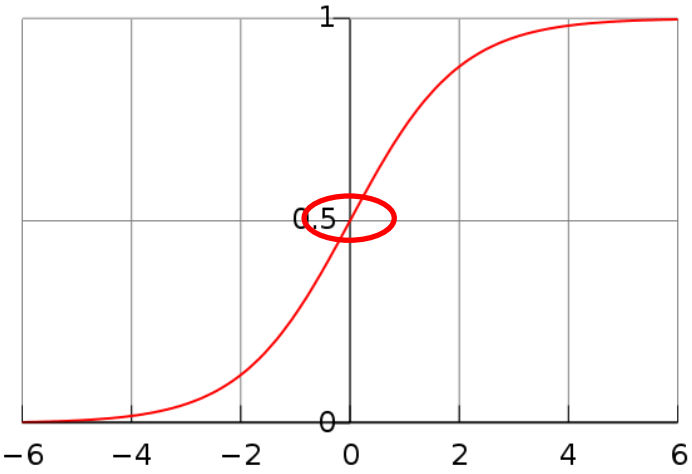
The Sigmoid Threshold Unit



Sigmoid Function

$$\sigma(y) = \frac{1}{1+e^{-y}}$$

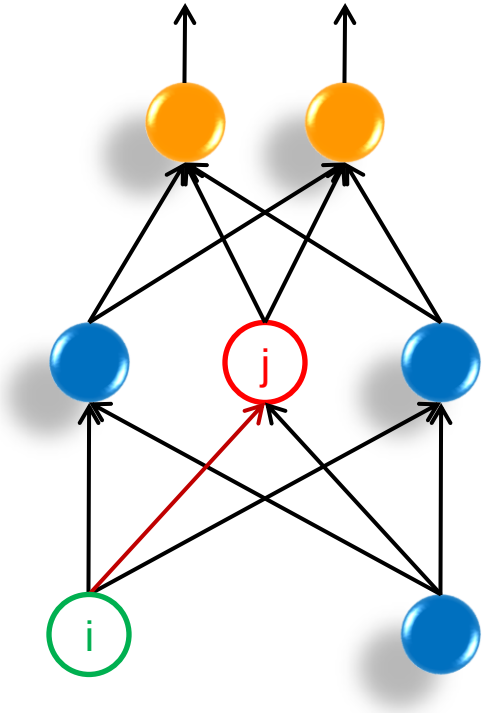
$$\frac{d\sigma(y)}{dy} = \sigma(y)\cdot(1-\sigma(y))$$



反向传播算法（BP算法）

$$E_d(\vec{w}) \equiv \frac{1}{2}\sum_{k \in outputs}(t_k - o_k)^2 \qquad \Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} x_{ji}$$



- x_{ji} = the ith input to unit j
- w_{ji} = the weight associated with the ith input to unit j
- $net_j = \sum w_{ji}x_{ji}$ (the weighted sum of inputs for unit j)
- o_j= the output of unit j
- t_j= the target output of unit j
- σ = the sigmoid function
- ***outputs*** = the set of units in the final layer
- ***Downstream*** (j) = the set of units directly taking the output of unit j as inputs



反向传播算法（BP算法）

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial o_j} \cdot \frac{\partial o_j}{\partial net_j}$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} x_{ji}$$

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} \sum_{k \in outputs} (t_k - o_k)^2$$

$$\frac{\partial o_j}{\partial net_j} = \frac{\partial \sigma(net_j)}{\partial net_j} = o_j(1 - o_j)$$

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} (t_j - o_j)^2$$

$$= \frac{1}{2} 2(t_j - o_j) \frac{\partial (t_j - o_j)}{\partial o_j}$$

$$= -(t_j - o_j)$$

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j) o_j (1 - o_j)$$

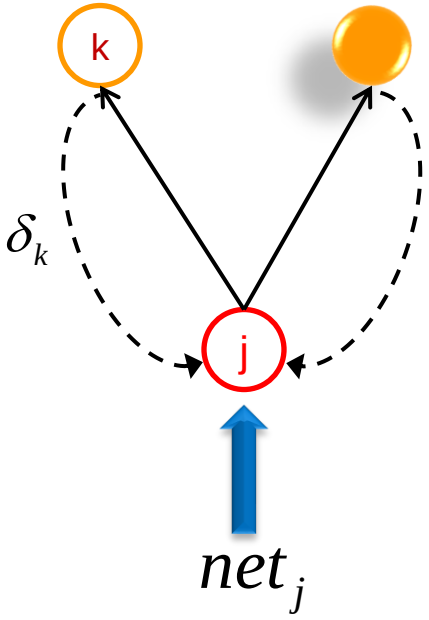
$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta (t_j - o_j) o_j (1 - o_j) x_{ji}$$



反向传播算法（BP算法）

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in Downstream(j)} \frac{\partial E_d}{\partial net_k} \frac{\partial net_k}{\partial net_j} = \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial o_j} \frac{\partial o_j}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k w_{kj} \frac{\partial o_j}{\partial net_j} = \sum_{k \in Downstream(j)} -\delta_k w_{kj} o_j (1 - o_j)$$



$$\delta_k = -\frac{\partial E_d}{\partial net_k}$$

$$\delta_j = o_j (1 - o_j) \sum_{k \in Downstream(j)} \delta_k w_{kj}$$

$$\Delta w_{ji} = \eta \delta_j x_{ji}$$



BP Framework

- BACKPROPAGATION (*training_examples*, η, n_{in}, n_{out}, n_{hidden})
- Create a network with n_{in} inputs, n_{hidden} hidden units and n_{out} output units.
- Initialize all network weights to **small** random numbers.
- Until the termination condition is met, Do
  - For each <$\boldsymbol{x}$, $\boldsymbol{t}$> in *training_examples*, Do
    - Input the instance $\boldsymbol{x}$ to the network and computer the output o of every unit.
    - For each **output** unit k, calculate its error term δ_k

$$\delta_k \leftarrow o_k(1-o_k)(t_k-o_k)$$

    - For each **hidden** unit h, calculate its error term δ_h

$$\delta_h \leftarrow o_h(1-o_h)\sum_{k\in outputs} w_{kh}\delta_k$$

    - Update each network weight w_{ji}

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji} = w_{ji} + \eta\delta_j x_{ji}$$



More about BP Networks …



- **Convergence and Local Minima**
  - **The search space is likely to be highly multimodal.**
  - **May easily get stuck at a local solution.**
  - **Need multiple trials with different initial weights.**
- **Evolving Neural Networks**
  - **Black-box optimization techniques (e.g., Genetic Algorithms)**
  - **Usually better accuracy**
  - **Can do some advanced training (e.g., structure + parameter).**
  - **Xin Yao (1999) "Evolving Artificial Neural Networks", *Proceedings of the IEEE*, pp. 1423-1447.**
- **Representational Power**
- **Deep Learning**



Evolving Artificial Neural Networks

XIN YAO, SENIOR MEMBER, IEEE

Invited Paper

Learning and evolution are two fundamental forms of adaptation. There has been a great interest in combining learning and evolution with artificial neural networks (ANN's) in recent years. This paper: 1) reviews different combinations between ANN's and evolutionary algorithms (EA's), including using EA's to evolve ANN connection weights, architectures, learning rules, and input features; 2) discusses different search operators which have been used in various EA's; and 3) points out possible future research directions. It is shown, through a considerably large literature review, that combinations between ANN's and EA's can lead to significantly better intelligent systems than relying on ANN's or EA's alone.

***Keywords*—Evolutionary computation, intelligent systems, neural networks.**

I. Introduction

Evolutionary artificial neural networks (EANN's) refer to a special class of artificial neural networks (ANN's) in which evolution is another fundamental form of adaptation in addition to learning [1]–[5]. Evolutionary algorithms (EA's) are used to perform various tasks, such as connection weight training, architecture design, learning rule adaptation, input feature selection, connection weight initialization, rule extraction from ANN's, etc. One distinct feature of EANN's is their adaptability to a dynamic environment. In other words, EANN's can adapt to an environment as well as changes in the environment. The two forms of adaptation, i.e., evolution and learning in EANN's, make their adaptation to a dynamic environment much more effective and efficient. In a broader sense, EANN's can be regarded as a general framework for adaptive systems, i.e., systems that can change their architectures and learning rules appropriately without human intervention.

This paper is most concerned with exploring possible benefits arising from combinations between ANN's and EA's. Emphasis is placed on the design of intelligent systems based on ANN's and EA's. Other combinations between ANN's and EA's for combinatorial optimization will be mentioned but not discussed in detail.

Manuscript received July 10, 1998; revised February 18, 1999. This work was supported in part by the Australian Research Council through its small grant scheme.

The author is with the School of Computer Science, University of Birmingham, Edgbaston, Birmingham B15 2TT U.K. (e-mail: xin@cs.bham.ac.uk).

Publisher Item Identifier S 0018-9219(99)06906-6.

A. Artificial Neural Networks

1) Architectures: An ANN consists of a set of processing elements, also known as neurons or nodes, which are interconnected. It can be described as a directed graph in which each node i performs a transfer function f_i of the form

$$y_i = f_i\left(\sum_{j=1}^{n} w_{ij}x_j - \theta_i\right) \tag{1}$$

where y_i is the output of the node i, x_j is the jth input to the node, and w_{ij} is the connection weight between nodes i and j. θ_i is the threshold (or bias) of the node. Usually, f_i is nonlinear, such as a heaviside, sigmoid, or Gaussian function.

ANN's can be divided into feedforward and recurrent classes according to their connectivity. An ANN is feedforward if there exists a method which numbers all the nodes in the network such that there is no connection from a node with a large number to a node with a smaller number. All the connections are from nodes with small numbers to nodes with larger numbers. An ANN is recurrent if such a numbering method does not exist.

In (1), each term in the summation only involves one input x_j. High-order ANN's are those that contain high-order nodes, i.e., nodes in which more than one input are involved in some of the terms of the summation. For example, a second-order node can be described as

$$y_i = f_i\left(\sum_{j,k=1}^{n} w_{ijk}x_jx_k - \theta_i\right)$$

where all the symbols have similar definitions to those in (1).

The architecture of an ANN is determined by its topological structure, i.e., the overall connectivity and transfer function of each node in the network.

0018–9219/99$10.00 © 1999 IEEE

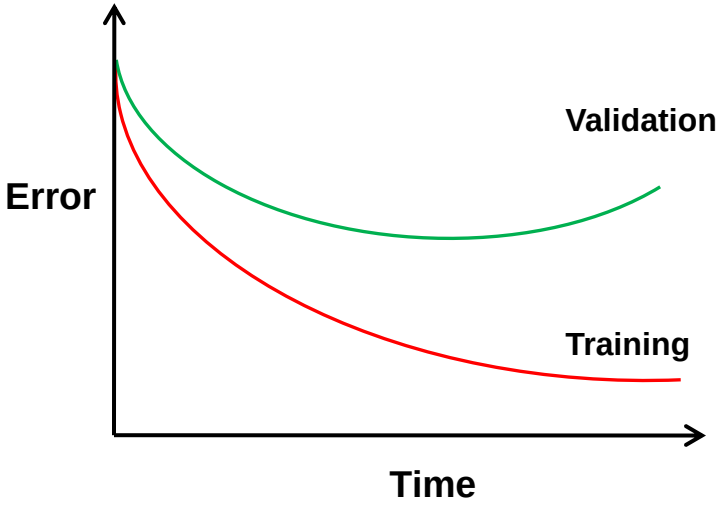
Xin Yao (1999) "Evolving Artificial Neural Networks", *Proceedings of the IEEE*, pp. 1423-1447.

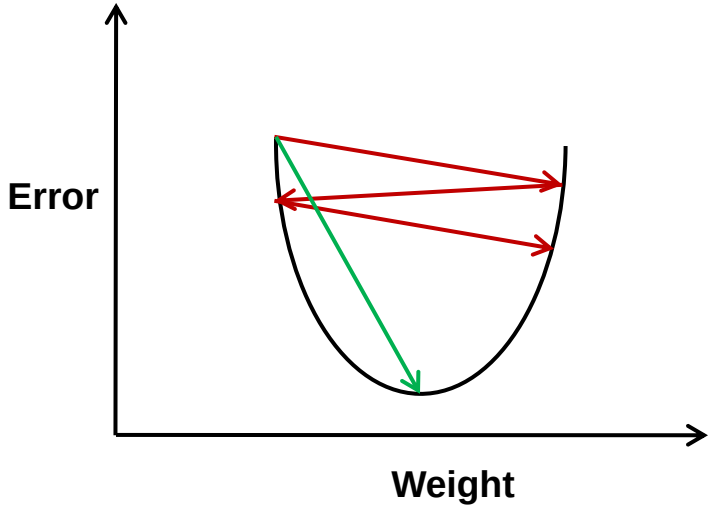


More about BP Networks …

- **Overfitting**
  - Tend to occur during later iterations.
  - Use validation dataset to terminate the training when necessary.
- **Practical Considerations**
  - Momentum
  - Adaptive learning rate
    - Small: slow convergence, easy to get stuck
    - Large: fast convergence, unstable

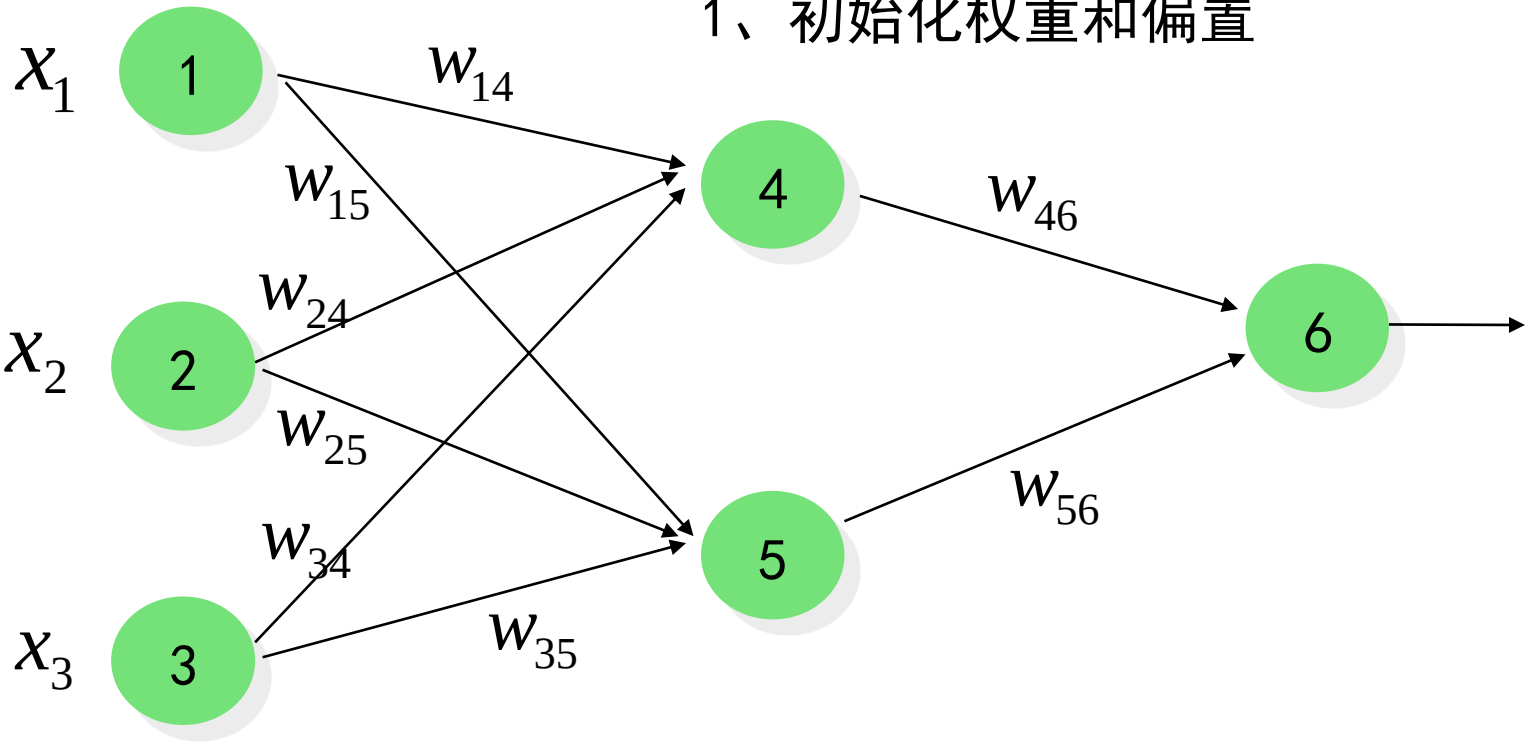
$$\Delta w_{ji}(n) = \eta \delta_j x_{ji} + \alpha \Delta w_{ji}(n-1)$$





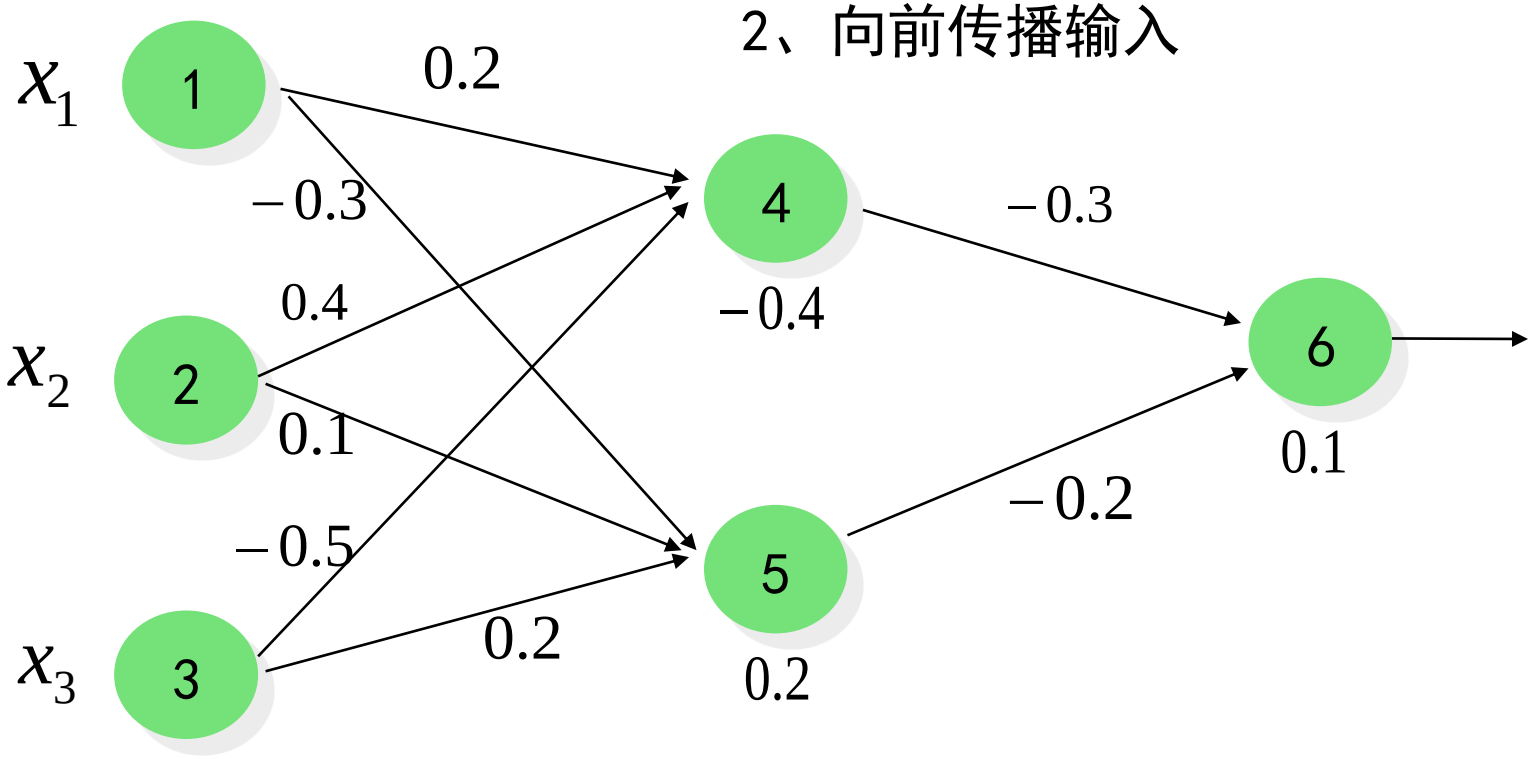
反向传播算法（BP算法）

1、初始化权重和偏置



反向传播算法（BP算法）

2、向前传播输入





反向传播算法（BP算法）

初始随机设置参数

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
1	0	1	0.2	−0.3	0.4	0.1	−0.5	0.2	−0.3	−0.2	−0.4	0.2	0.1

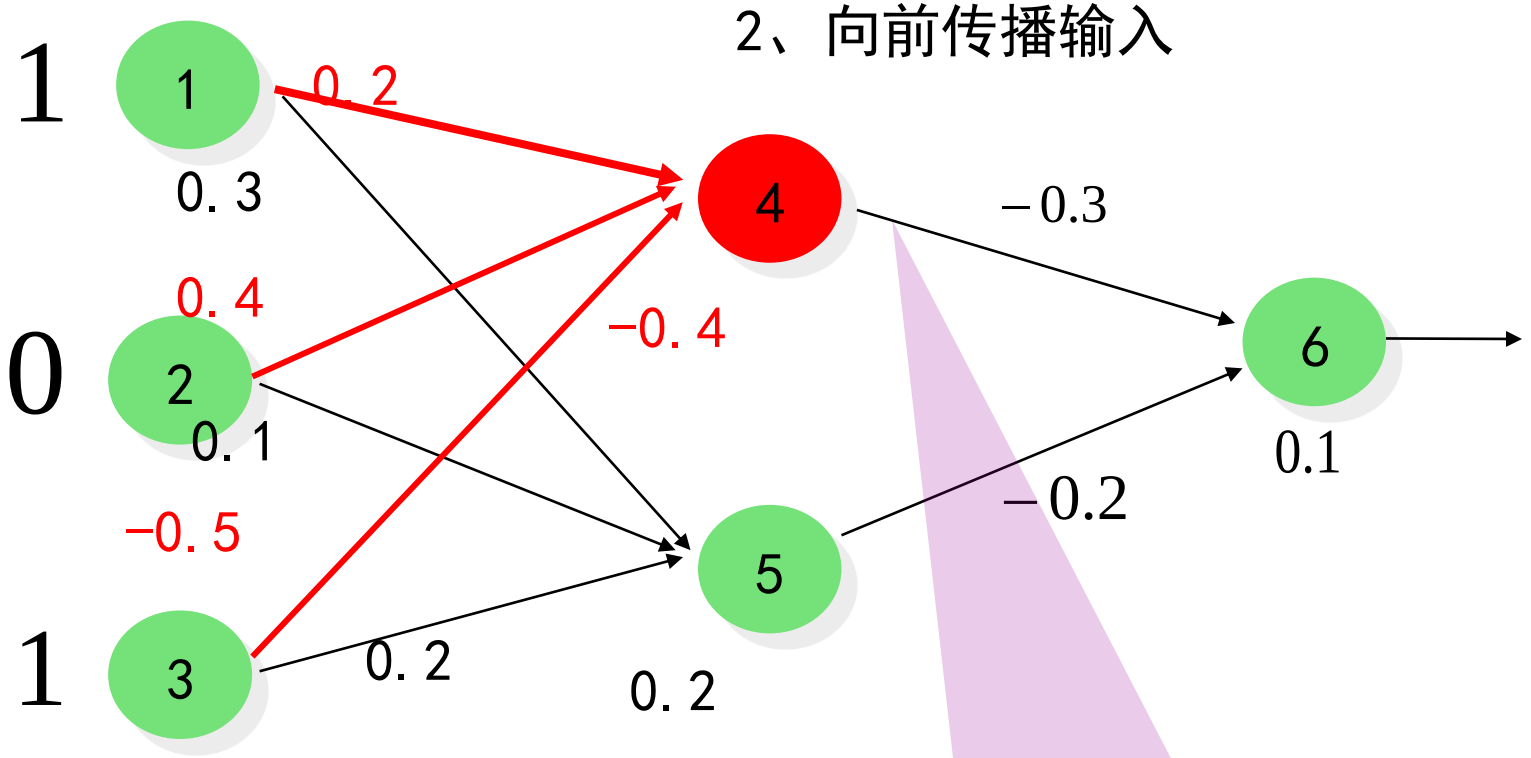
静输入和输出的表格

Unit j	Net input, I_j	Output, O_j
4	$0.2+0-0.5-0.4=-0.7$	$1/(1+e^{0.7})=0.332$
5	$-0.3+0+0.2+0.2=0.1$	$1/(1+e^{-0.1})=0.525$
6	$(-0.3)(0.332)-(0.2)(0.525)+0.1=-0.105$	$1/(1+e^{0.105})=0.474$

人工神经网络

后向传播算法

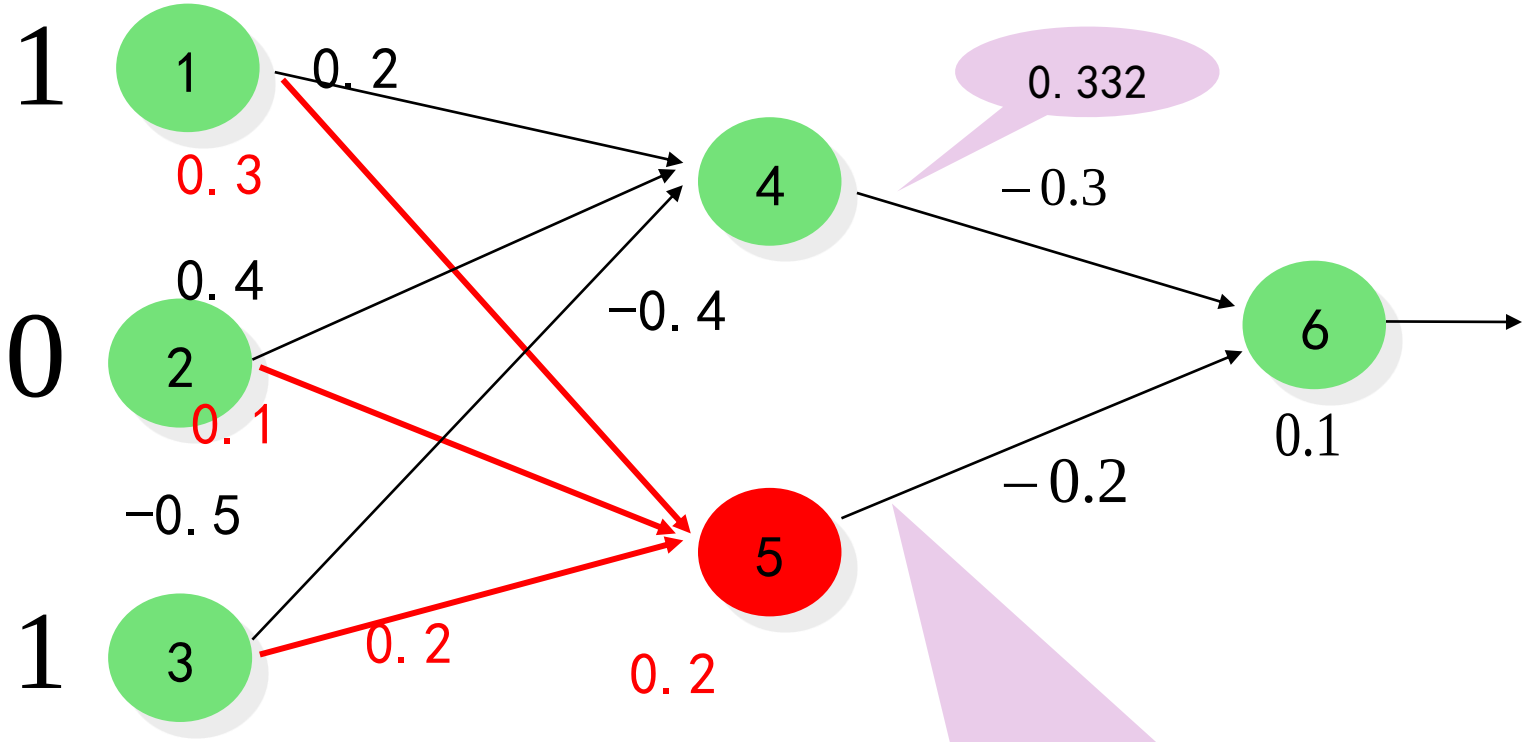
2、向前传播输入



0.2 + 0 − 0.5 − 0.4 = −0.7

输出：1/(1+e^0.7)=0.332

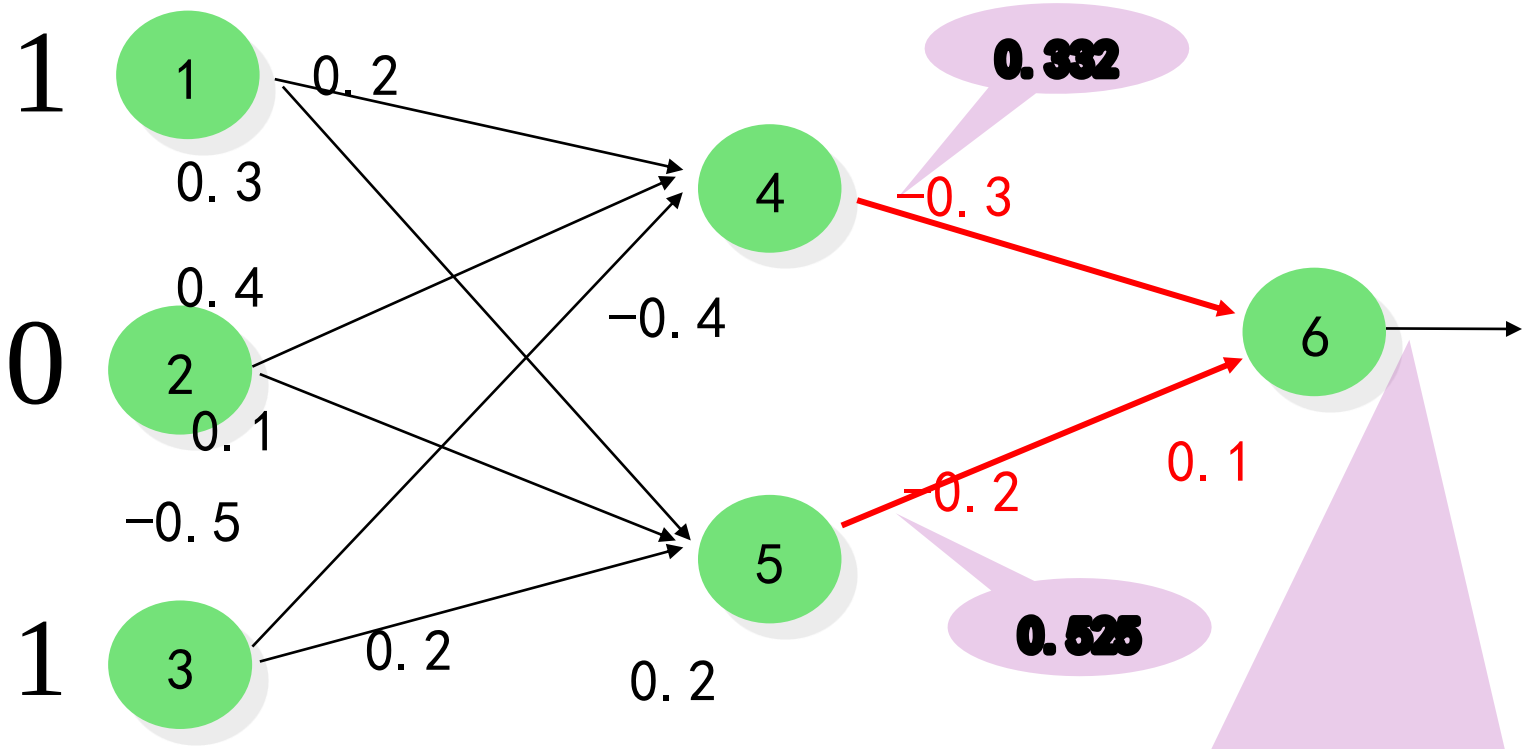
反向传播算法（BP算法）



−0.3 + 0 + 0.2 + 0.2 = 0.1

输出：1/(1+e^−0.1)=0.525

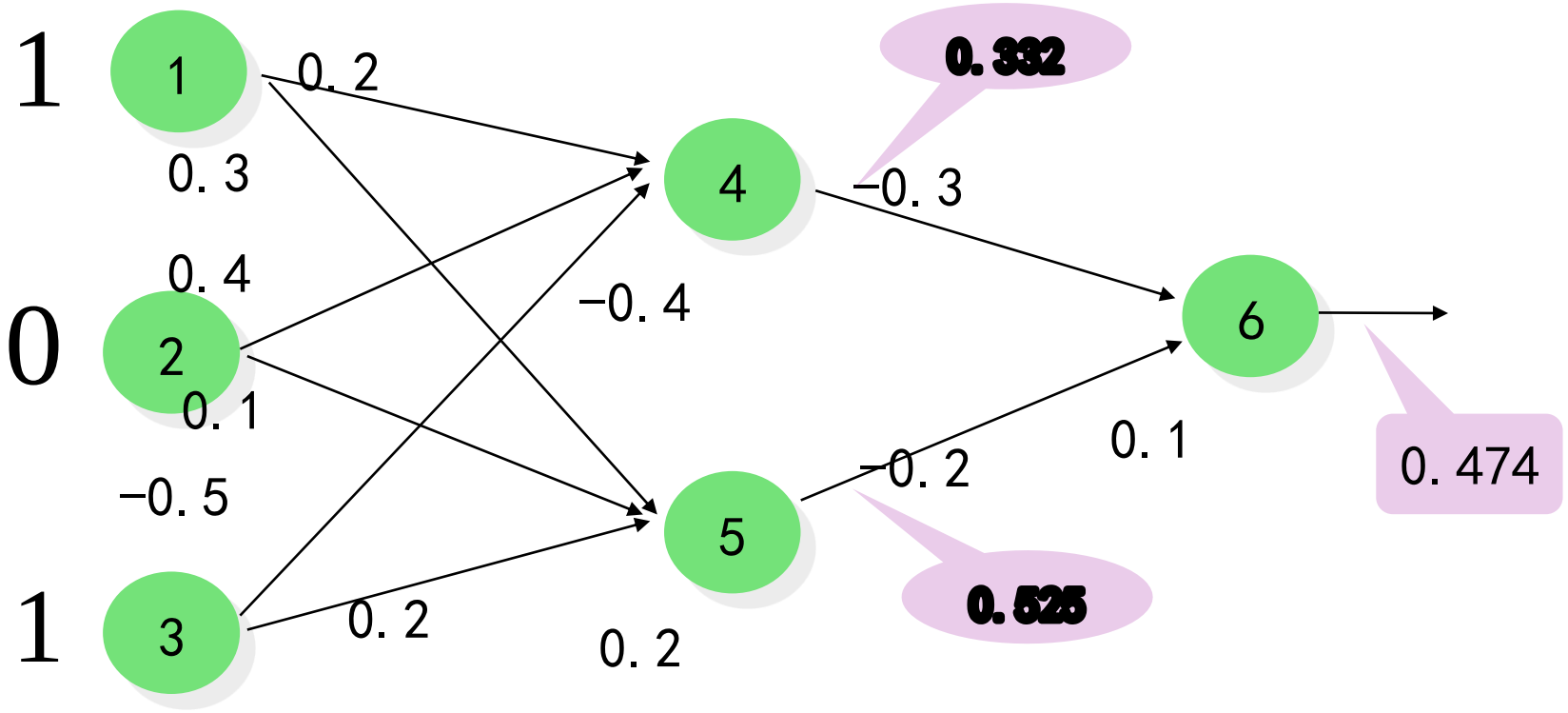
反向传播算法（BP算法）



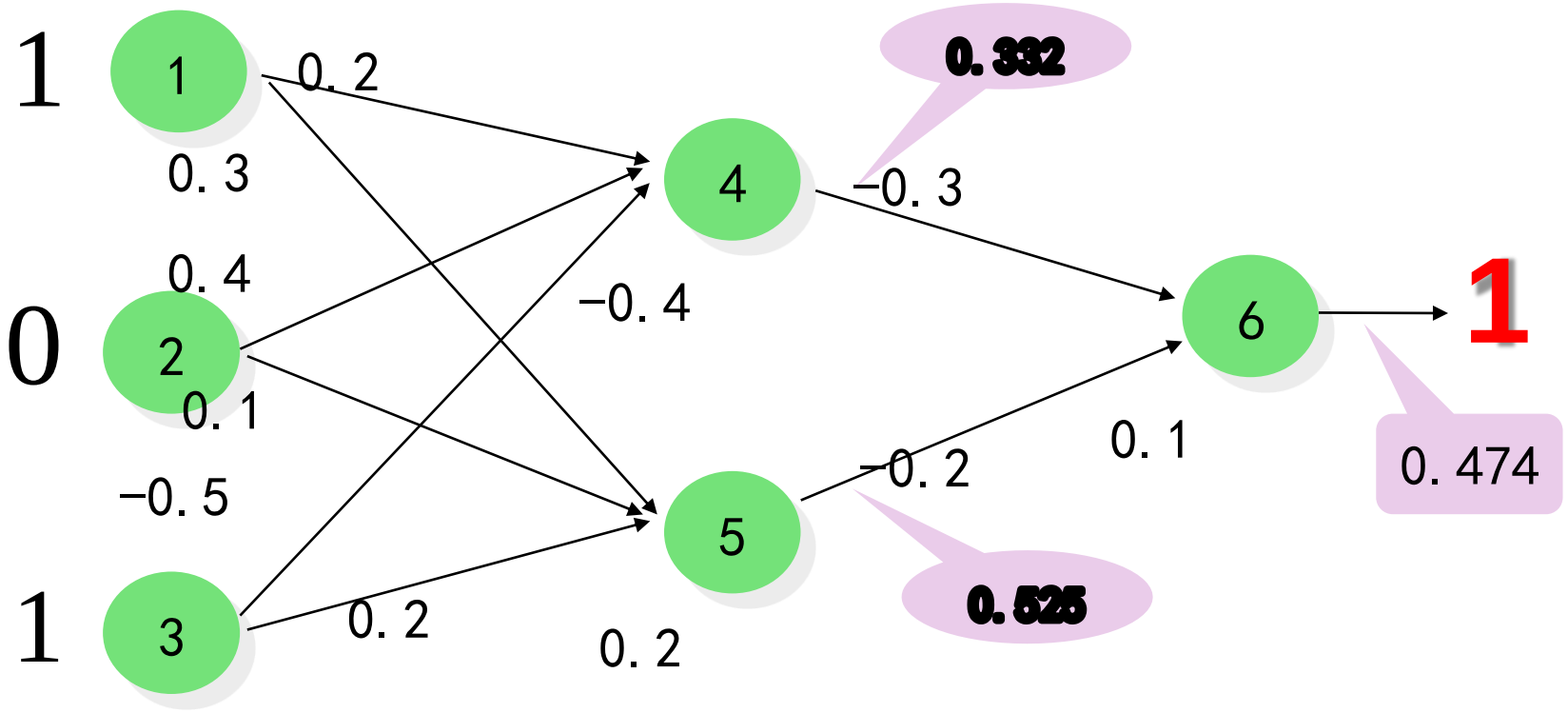
(−0.3)(0.332)−(0.2)(0.525)+0.1= **−0.105**

输出：1/(1+e^−0.105)=0.474

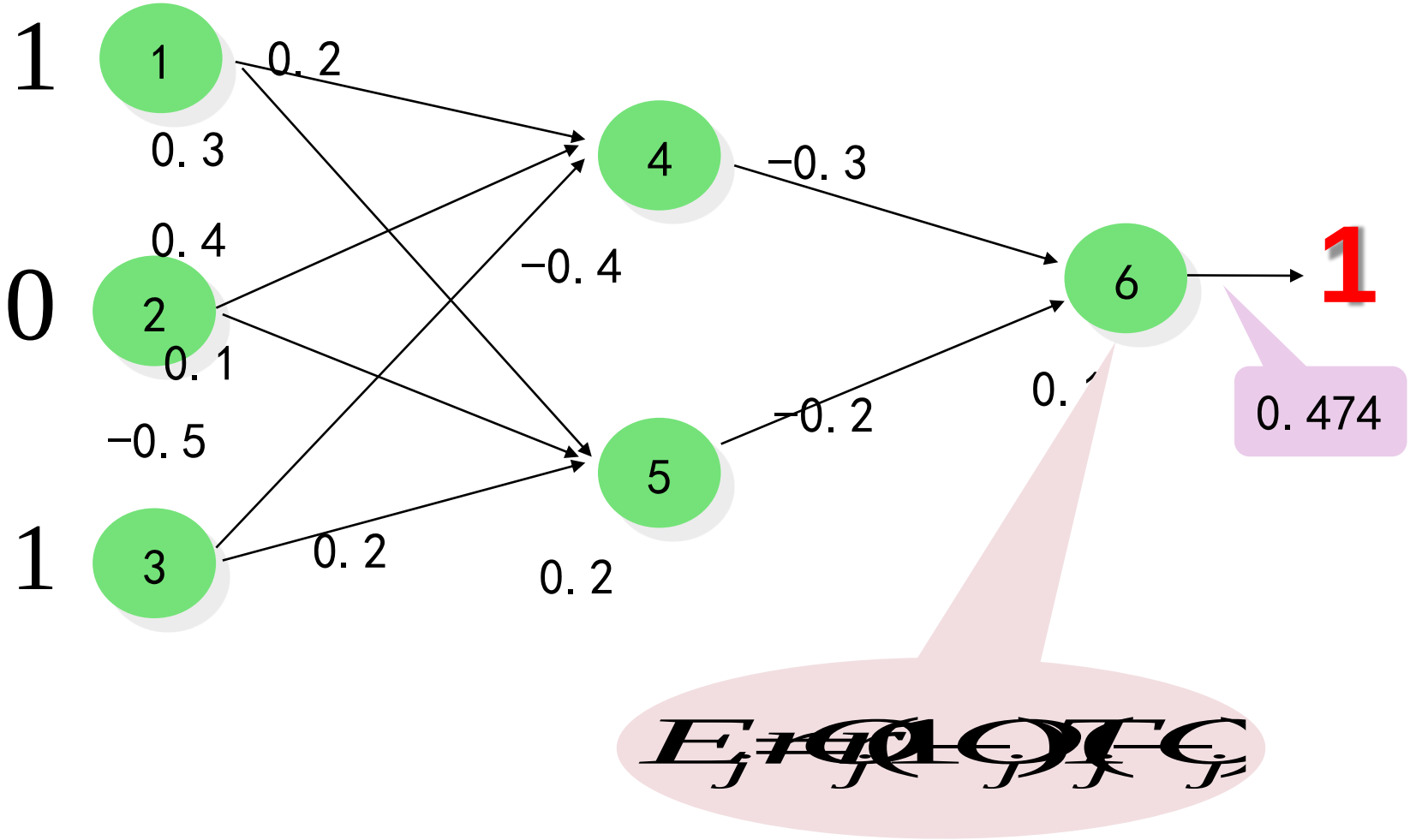
反向传播算法（BP算法）



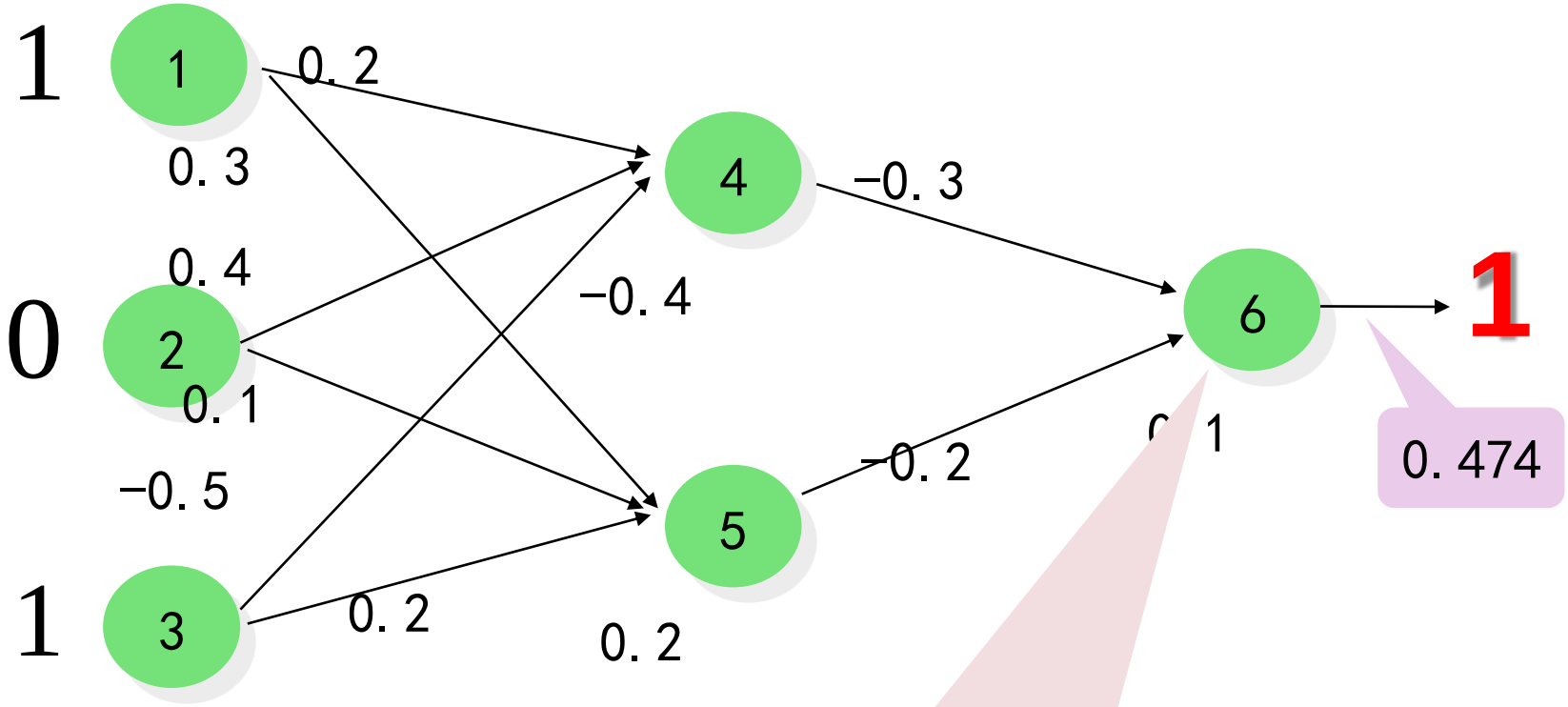
反向传播算法（BP算法）



反向传播算法（BP算法）

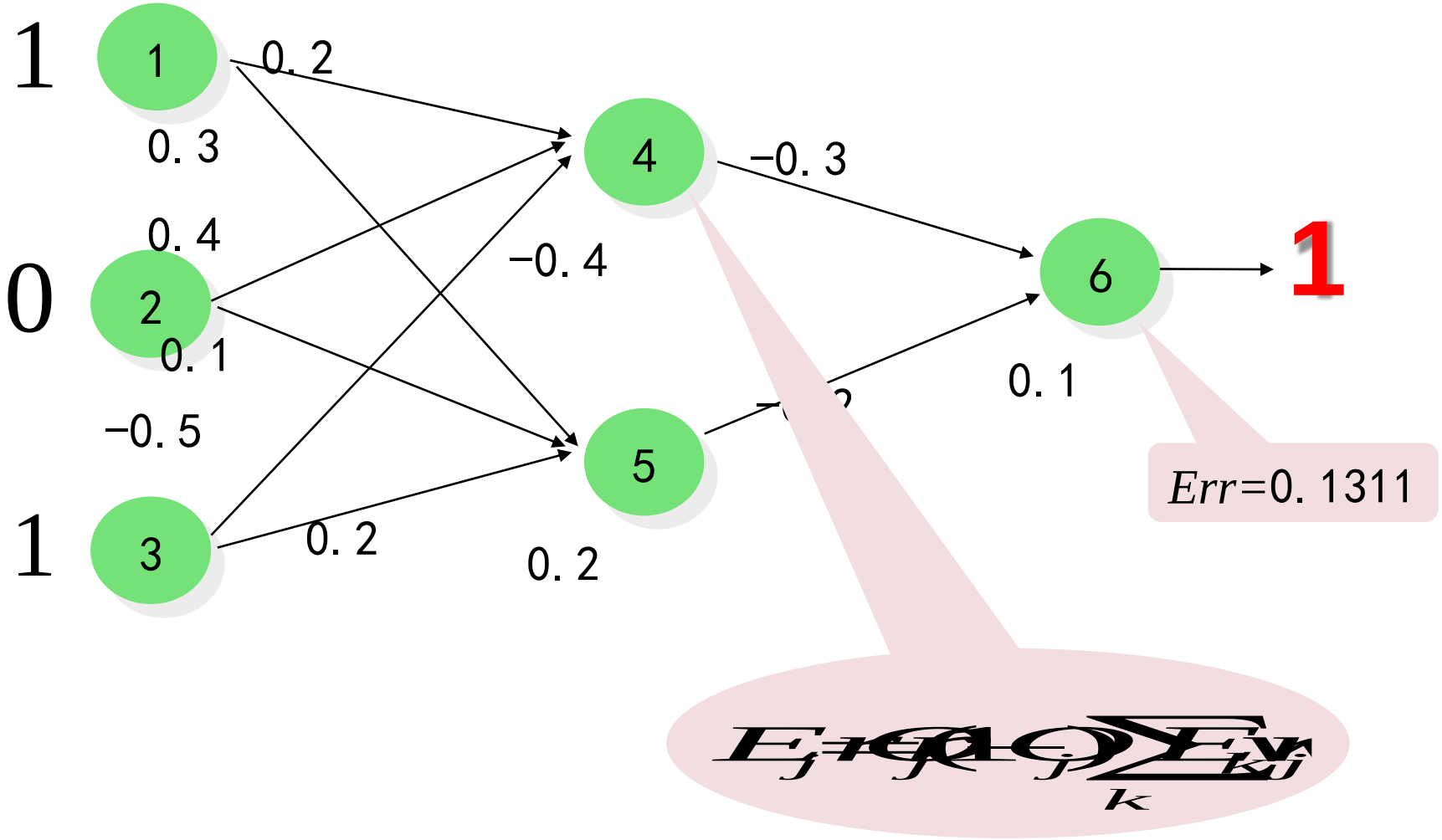


反向传播算法（BP算法）

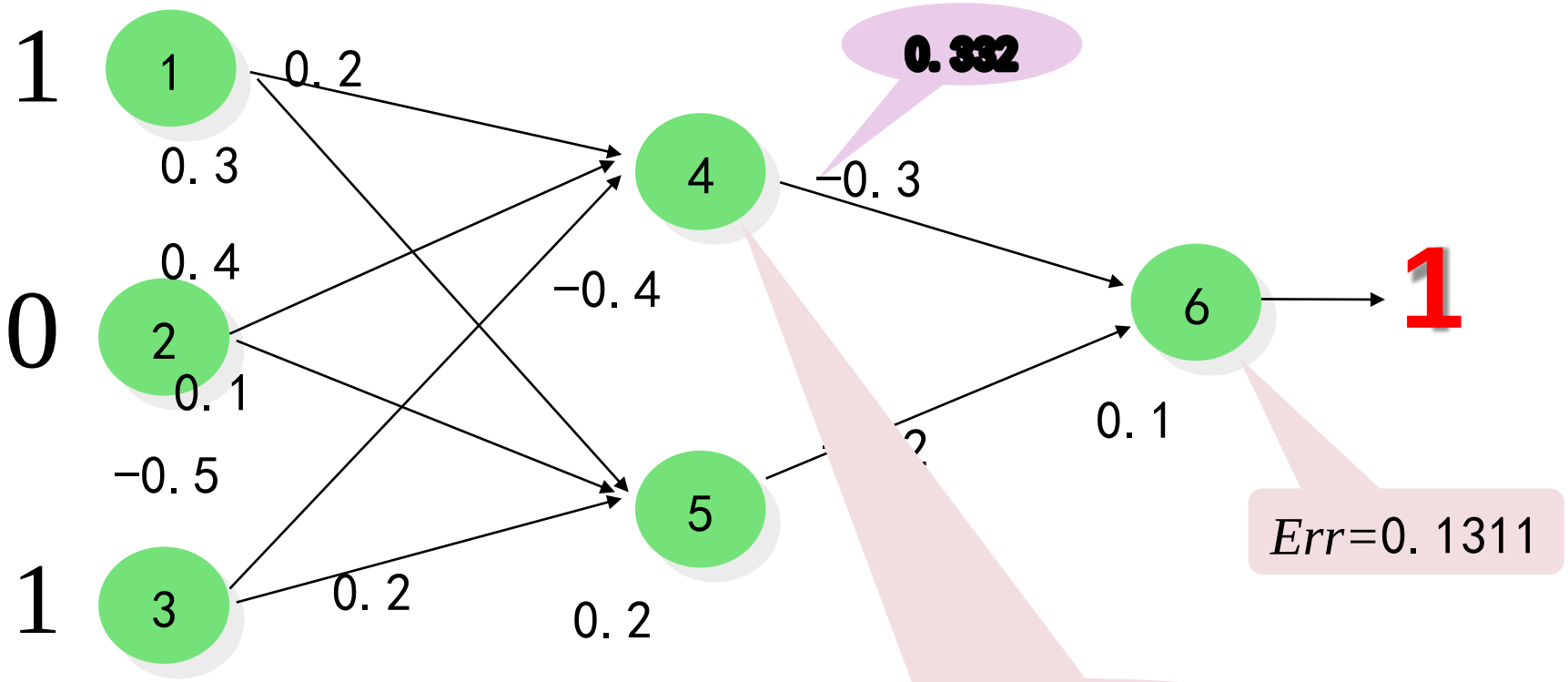


$$Err = 0.474(1-0.474)(1-0.474) = 0.1311$$

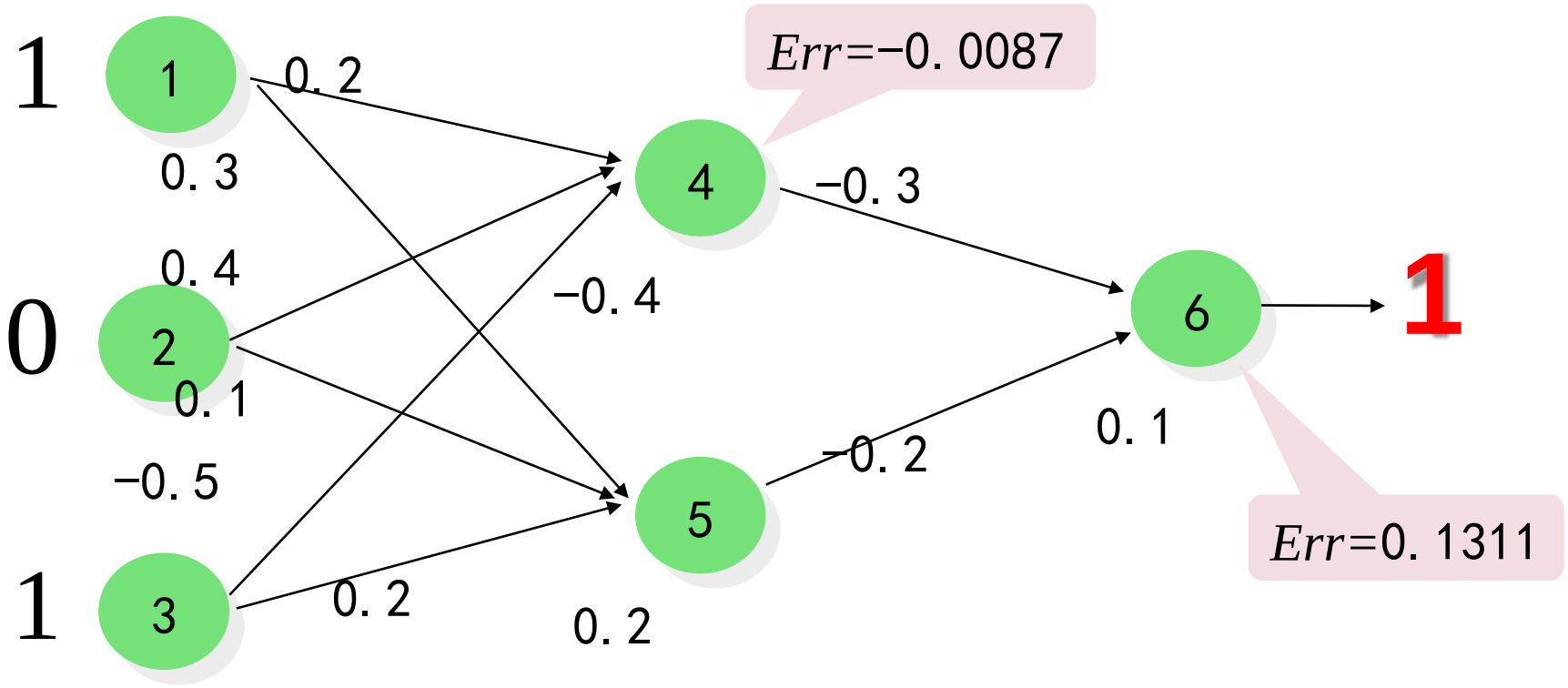
反向传播算法（BP算法）



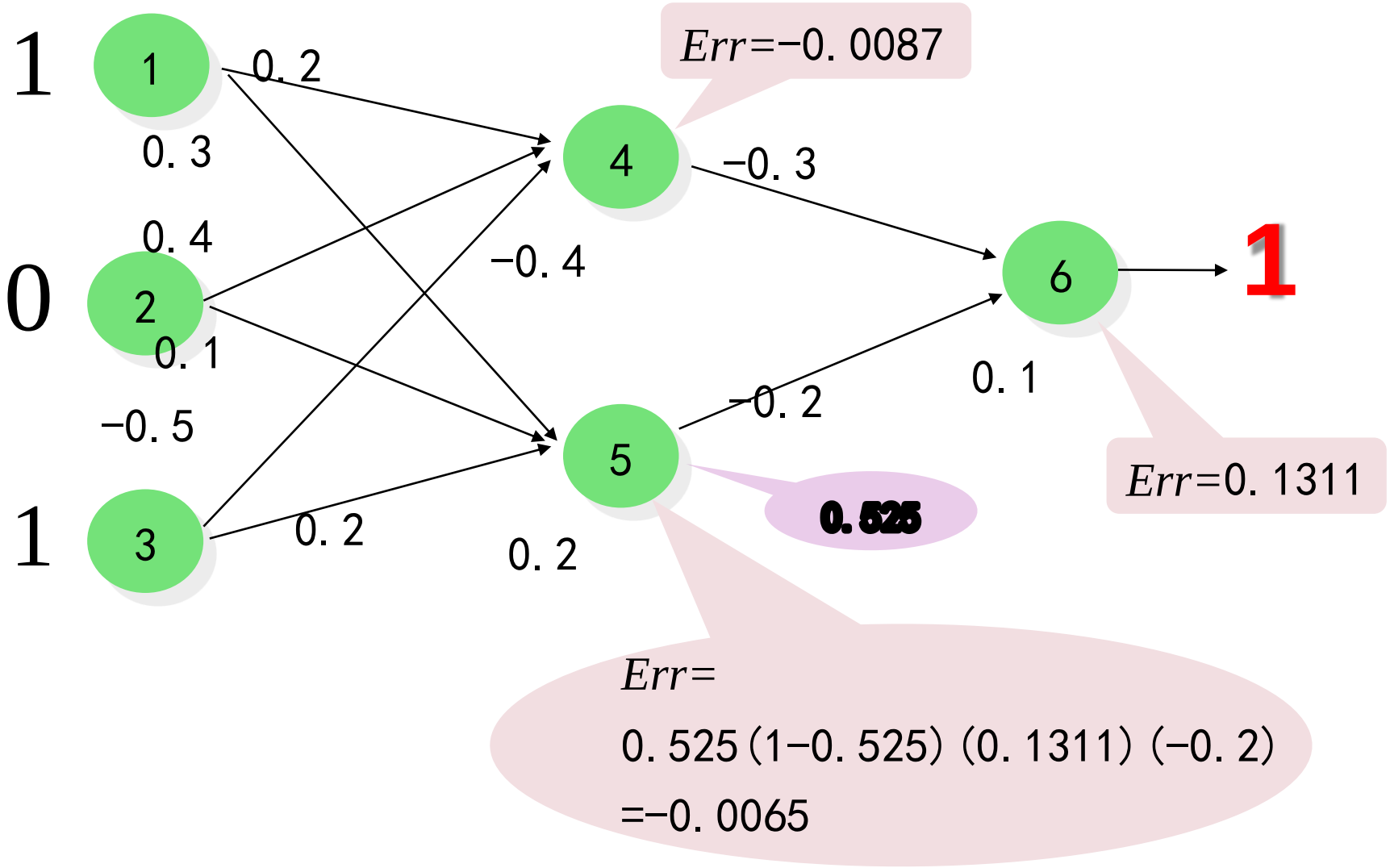
反向传播算法（BP算法）


1
0
1
1
2
3
4
5
6
0.2
0.3
0.4
0.1
−0.5
0.2
−0.4
0.2
0.332
−0.3
0.1
1
Err=0.1311
Err=
0.332(1−0.332)(0.1311)(−0.3)
=−0.0087

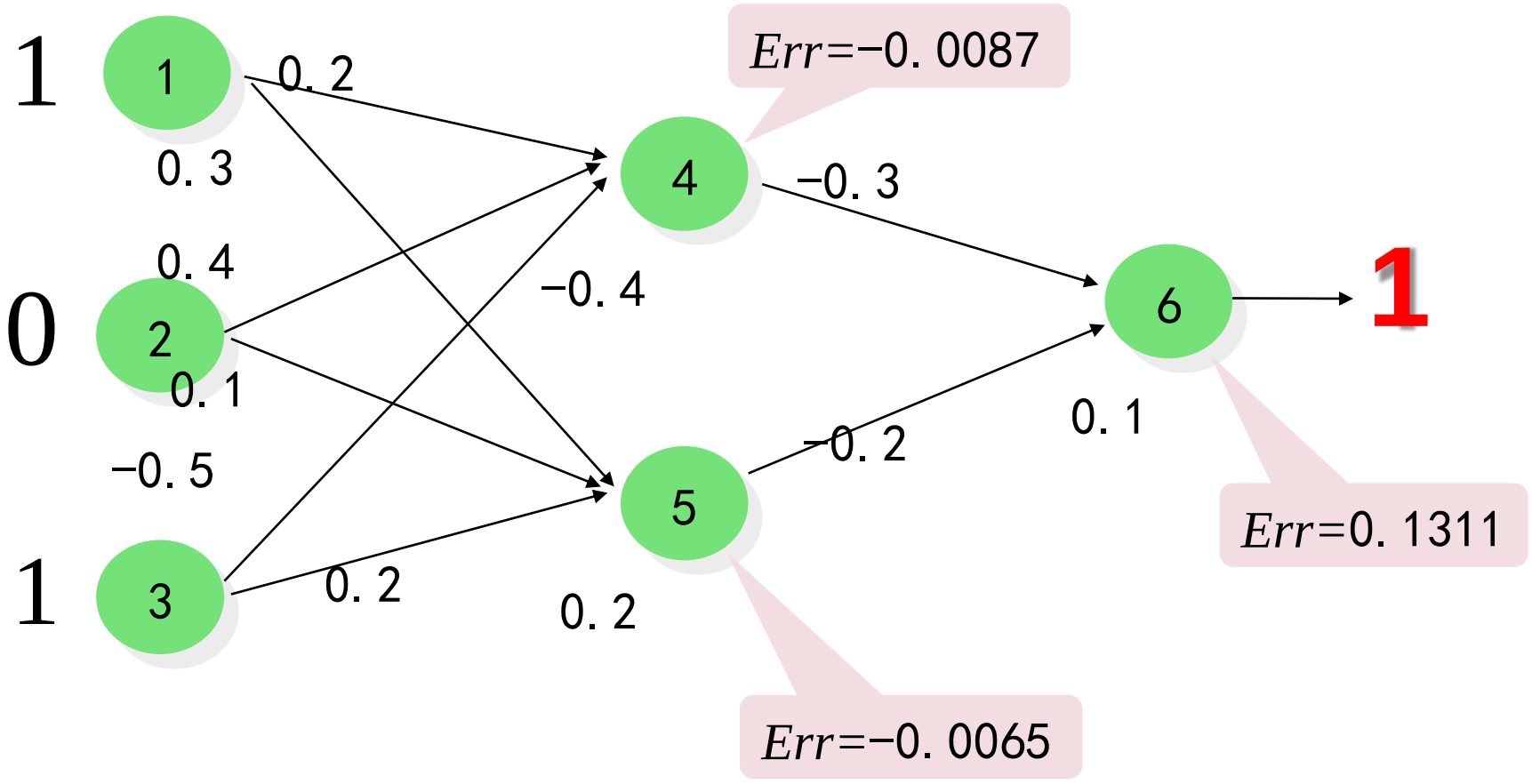
反向传播算法（BP算法）



反向传播算法（BP算法）


1
0
1
1
2
3
4
5
6
0.2
0.3
0.4
0.1
−0.5
−0.4
0.2
0.2
−0.3
−0.2
0.1
1
Err=−0.0087
0.525
Err=0.1311
Err=
0.525(1−0.525)(0.1311)(−0.2)
=−0.0065

反向传播算法（BP算法）


1
0
1
1
2
3
4
5
6
0.2
0.3
0.4
0.1
−0.5
−0.4
0.2
0.2
−0.3
−0.2
0.1
1
Err=−0.0087
Err=0.1311
Err=−0.0065

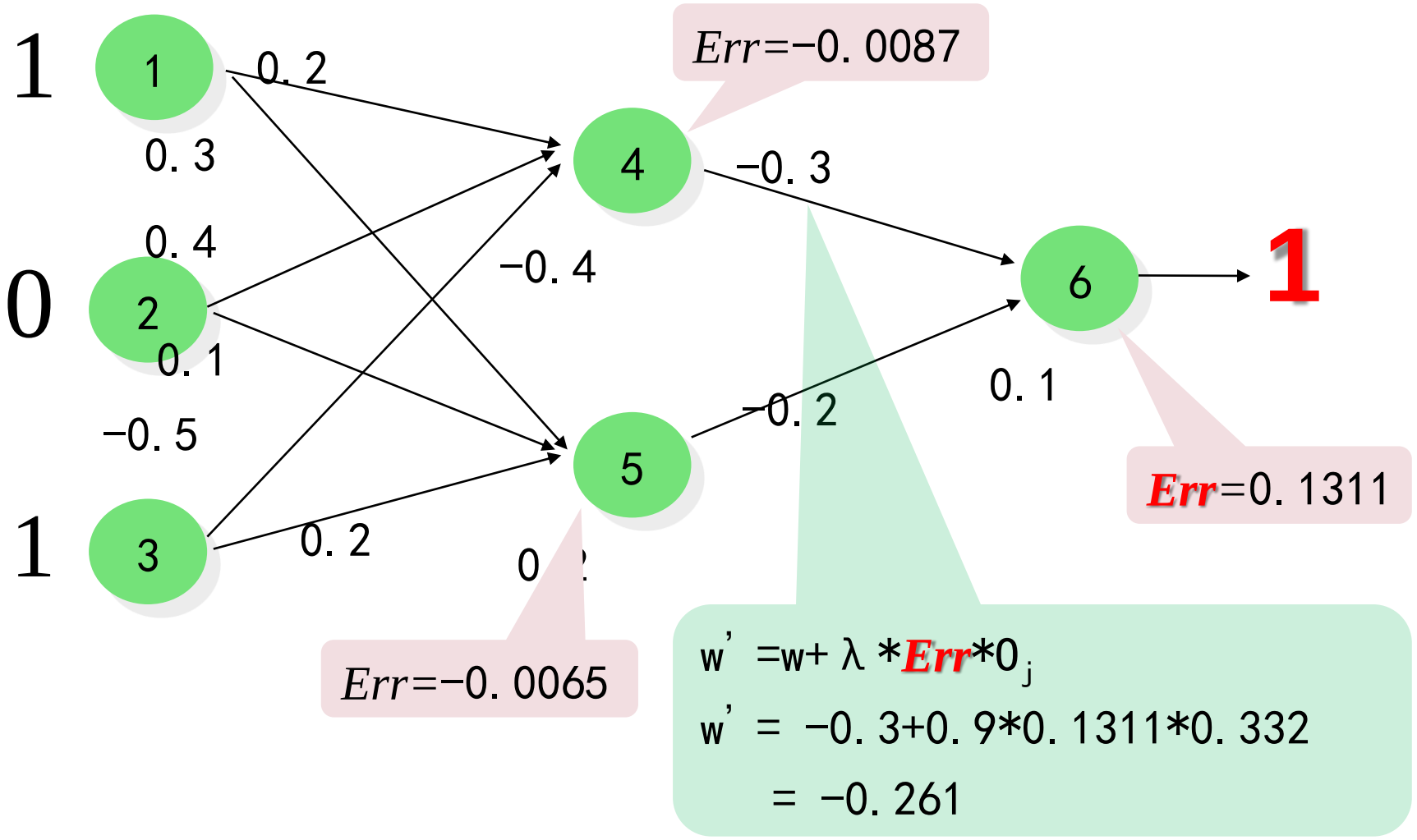


反向传播算法（BP算法）

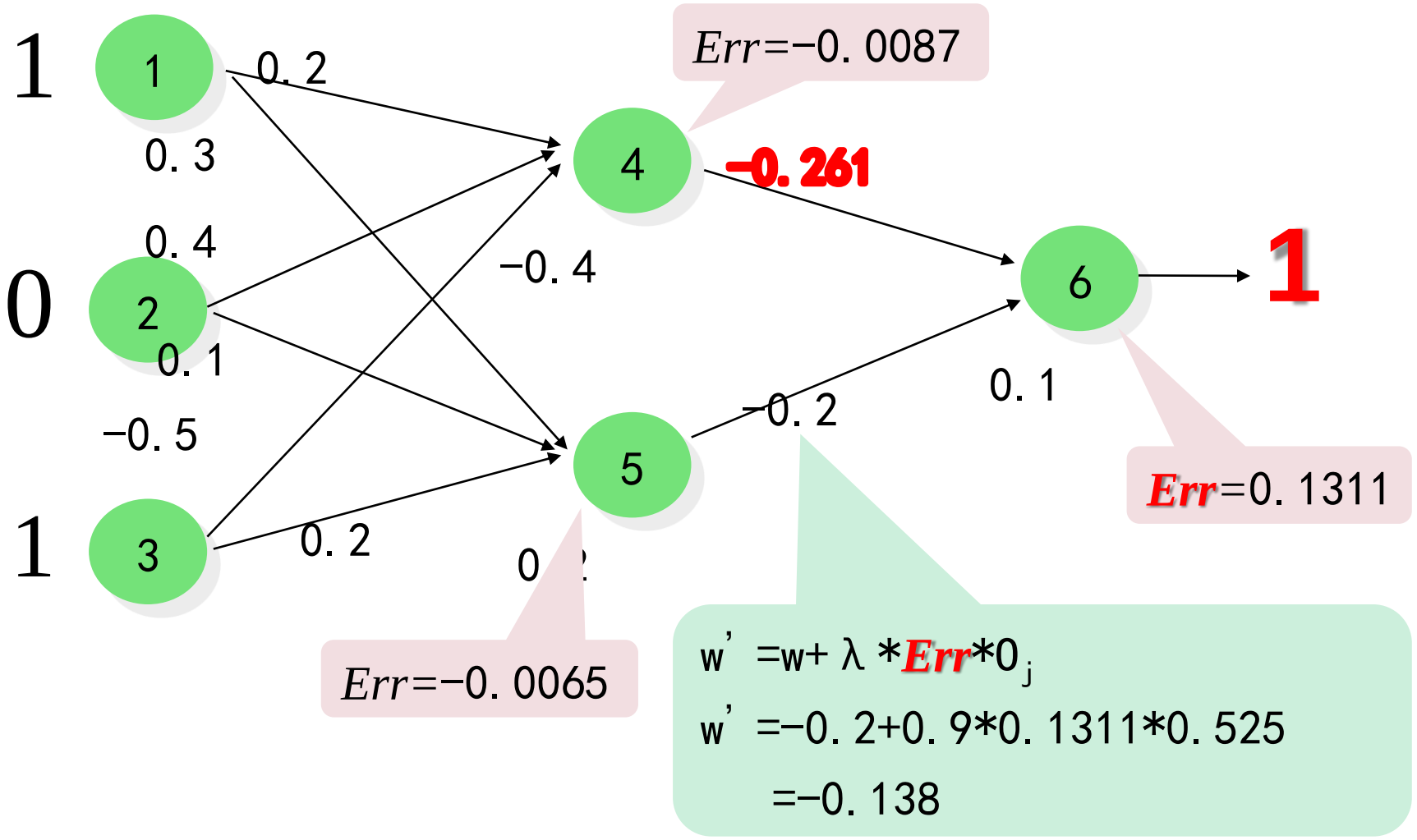
计算各结点的误差

Unit j	Err_j
6	$(0.474)(1-0.474)(1-0.474)=0.1311$
5	$(0.525)(1-0.525)(0.1311)(-0.2)=-0.0065$
4	$(0.332)(1-0.332)(0.1311)(-0.3)=-0.0087$

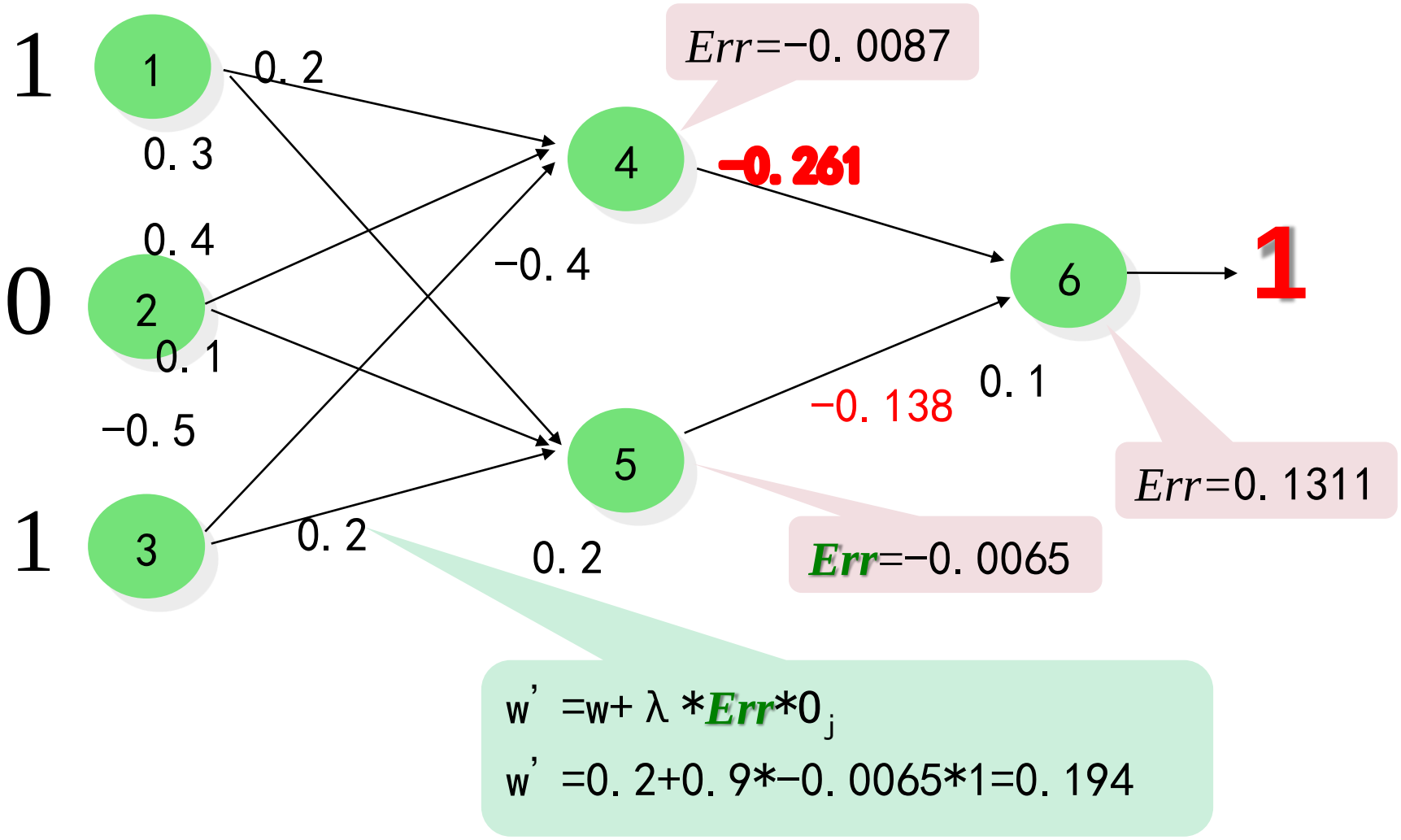
反向传播算法（BP算法）


1
0
1
1
2
3
4
5
6
0. 2
0. 3
0. 4
0. 1
−0. 5
−0. 4
0. 2
−0. 3
−0. 2
0. 1
1
Err=−0. 0087
Err=−0. 0065
Err=0. 1311
w' =w+ λ *Err*0j
w' = −0. 3+0. 9*0. 1311*0. 332
= −0. 261

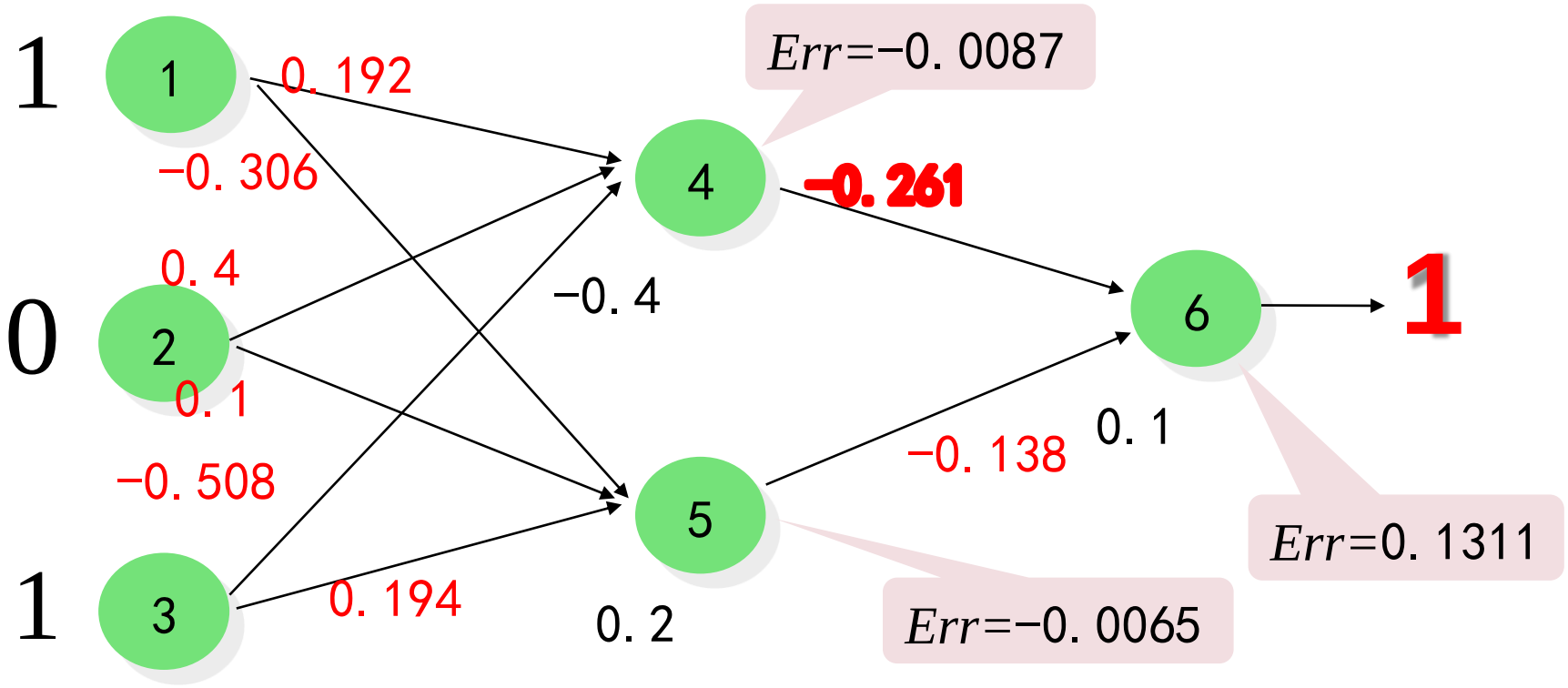
反向传播算法（BP算法）


1
0
1
1
2
3
4
5
6
0. 2
0. 3
0. 4
0. 1
−0. 5
−0. 4
0. 2
0. 2
0. 1
−0. 2
Err=−0. 0087
−0. 261
1
Err=0. 1311
Err=−0. 0065
w’ =w+ λ *Err*0j
w’ =−0. 2+0. 9*0. 1311*0. 525
=−0. 138

反向传播算法（BP算法）


1
0
1
1
2
3
4
5
6
0.2
0.3
0.4
0.1
−0.5
−0.4
0.2
0.2
0.1
Err=−0.0087
−0.261
−0.138
Err=0.1311
Err=−0.0065
1
w’ =w+ λ *Err*0j
w’ =0.2+0.9*−0.0065*1=0.194

反向传播算法（BP算法）


1
0
1
1
2
3
4
5
6
0.192
-0.306
0.4
0.1
-0.508
0.194
-0.4
0.2
-0.261
-0.138
0.1
Err=-0.0087
Err=-0.0065
Err=0.1311
1



反向传播算法（BP算法）

如此反复迭代地计算输出误差，根据误差反向传播反馈并计算和调整新的权重和偏置

Weight or bias	New value
w_{46}	$-0.3+(0.9)(0.1311)(0.332)=-0.261$
w_{56}	$-0.2+(0.9)(0.1311)(0.525)=-0.138$
w_{14}	$0.2+(0.9)(-0.0087)(1)=0.192$
w_{15}	$-0.3+(0.9)(-0.0065)(1)=-0.306$
w_{24}	$0.4+(0.9)(-0.0087)(0)=0.4$
w_{25}	$0.1+(0.9)(-0.0065)(0)=0.1$
w_{34}	$-0.5+(0.9)(-0.0087)(1)=-0.508$
w_{35}	$0.2+(0.9)(-0.0065)(1)=0.194$
θ_6	$0.1+(0.9)(0.1311)=0.218$
θ_5	$0.2+(0.9)(-0.0065)=0.194$
θ_4	$-0.4+(0.9)(-0.0087)=-0.408$



神经网络优缺点

- 神经网络最早是由心理学家和神经学家提出的，旨在寻求开发和测试神经的计算模拟
- 神经网络是一组连接的输入/输出单元，其中每个连接都与一个权相关联
- 在学习阶段，通过调整神经网络的权值，使能够预测输入样本的正确类标记
- 神经网络的优点
  - 生物神经系统的信息处理能力来自于在许多神经元上分布的高度并行计算过程
  - 对噪音数据的高承受能力
  - 对未知样本的分类能力
- 神经网络缺点
  - 需要很长的训练时间，因而对于有足够长训练时间的应用更合适
  - 很难解释蕴涵在学习权之中的符号含义
  - 它需要大量的参数，这些通常主要靠经验确定，如网络拓扑或“结构”